

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего  
образования

**"Удмуртский государственный университет"**

Институт математики, информационных технологий и физики

## **КУРСОВАЯ РАБОТА**

по дисциплине "Машинная графика"

**Таблица дробных производных**

**Грюнвальда–Летникова элементарных функций**

Выполнил:

студент группы ОБ-01.03.02.02-31

направления 01.03.02

"Прикладная математика и информатика"

**Шестак Александр Сергеевич**

Проверил:

к.ф.-м.н., доцент

**Банников Александр Сергеевич**

Ижевск, 2026

# Содержание

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>Введение</b>                                                                  | <b>5</b>  |
| <b>1 Теоретические основы дробного дифференцирования</b>                         | <b>7</b>  |
| 1.1 От обычной производной к производной дробного порядка .                      | 7         |
| 1.2 Почему дробную производную нельзя записать короткой раз-<br>ностью . . . . . | 8         |
| 1.3 Левый вариант производной . . . . .                                          | 8         |
| 1.4 Подходы к дробной производной и выбор схемы . . . . .                        | 9         |
| 1.5 Определение Грюнвальда–Летникова . . . . .                                   | 10        |
| 1.6 Смысл коэффициентов . . . . .                                                | 10        |
| 1.7 Память метода и длина истории . . . . .                                      | 11        |
| 1.8 Эталонные формулы для проверки . . . . .                                     | 12        |
| 1.9 Что из теории используется дальше . . . . .                                  | 12        |
| <b>2 Метод Грюнвальда–Летникова как численный алгоритм</b>                       | <b>14</b> |
| 2.1 От определения к расчёту . . . . .                                           | 14        |
| 2.2 Точная производная и численная схема . . . . .                               | 14        |
| 2.3 Режимы вычисления: нижний предел и длина истории . . .                       | 14        |
| 2.4 Расчёт коэффициентов без факториалов . . . . .                               | 15        |
| 2.5 Схема вычисления одного значения . . . . .                                   | 16        |
| 2.6 Пример ручного расчёта . . . . .                                             | 16        |
| 2.7 Нормировка на $h^\alpha$ . . . . .                                           | 16        |
| 2.8 Параметры $\alpha$ , $h$ , $N$ . . . . .                                     | 17        |
| 2.9 Краевой эффект и область определения . . . . .                               | 17        |
| 2.10 Трудоёмкость и источники ошибки . . . . .                                   | 18        |
| 2.11 Способы оценки ошибки . . . . .                                             | 18        |
| <b>3 Программная реализация</b>                                                  | <b>19</b> |
| 3.1 Выбор инструментов . . . . .                                                 | 19        |
| 3.2 Структура проекта . . . . .                                                  | 20        |
| 3.3 Коэффициенты и производная . . . . .                                         | 20        |
| 3.4 Формирование таблиц и графиков . . . . .                                     | 21        |
| 3.5 Запуск . . . . .                                                             | 21        |
| 3.6 Веб-интерфейс . . . . .                                                      | 22        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 3.7      | Тестирование программы . . . . .                                   | 23        |
| <b>4</b> | <b>Вычислительные эксперименты</b>                                 | <b>25</b> |
| 4.1      | Сводная таблица дробных производных элементарных функций . . . . . | 25        |
| 4.2      | План экспериментов . . . . .                                       | 27        |
| 4.3      | Изменение функции при росте порядка производной . . . . .          | 28        |
| 4.4      | Проверка на целых порядках . . . . .                               | 30        |
| 4.5      | Влияние шага $h$ . . . . .                                         | 31        |
| 4.6      | Влияние числа членов $N$ . . . . .                                 | 32        |
| 4.7      | Память метода . . . . .                                            | 33        |
| 4.8      | Сходимость и порядок точности . . . . .                            | 33        |
| 4.9      | Степенные функции и логарифм . . . . .                             | 35        |
| 4.10     | Экспонента, левая и правая схемы, касательная . . . . .            | 37        |
| 4.11     | Карта дробных производных . . . . .                                | 39        |
| <b>5</b> | <b>Точность, устойчивость и пути её повышения</b>                  | <b>39</b> |
| 5.1      | Аналитическая оценка порядка аппроксимации . . . . .               | 39        |
| 5.2      | Проверка через сумму коэффициентов . . . . .                       | 41        |
| 5.3      | Порядок усечения по числу членов . . . . .                         | 42        |
| 5.4      | Предел точности и порог округления . . . . .                       | 43        |
| 5.5      | Повышение порядка: экстраполяция Ричардсона . . . . .              | 44        |
| 5.6      | Нижний предел против конечной памяти . . . . .                     | 46        |
| 5.7      | Совместное влияние шага и числа членов . . . . .                   | 46        |
| 5.8      | Сдвинутая схема Грюнвальда . . . . .                               | 46        |
| 5.9      | Сравнение с производной Капуто . . . . .                           | 50        |
|          | <b>Заключение</b>                                                  | <b>52</b> |
|          | <b>Список литературы</b>                                           | <b>54</b> |
|          | <b>Приложение А Структура проекта и запуск</b>                     | <b>56</b> |
|          | <b>Приложение Б Основные листинги программы</b>                    | <b>57</b> |
|          | Б.1 fractional.py . . . . .                                        | 57        |
|          | Б.2 functions.py . . . . .                                         | 59        |

|      |                                             |    |
|------|---------------------------------------------|----|
| Б.3  | experiments.py . . . . .                    | 62 |
| Б.4  | tables.py . . . . .                         | 68 |
| Б.5  | plots.py . . . . .                          | 70 |
| Б.6  | main.py . . . . .                           | 78 |
| Б.7  | web/src/domain/fractional.ts . . . . .      | 79 |
| Б.8  | web/src/domain/derivativeModel.ts . . . . . | 82 |
| Б.9  | web/src/App.tsx . . . . .                   | 85 |
| Б.10 | tests/conftest.py . . . . .                 | 86 |
| Б.11 | tests/test_coefficients.py . . . . .        | 86 |
| Б.12 | tests/test_fractional_orders.py . . . . .   | 87 |
| Б.13 | tests/test_function_domains.py . . . . .    | 88 |
| Б.14 | tests/test_integer_orders.py . . . . .      | 88 |
| Б.15 | tests/test_alt_schemes.py . . . . .         | 89 |
| Б.16 | tests/test_master_table.py . . . . .        | 90 |

|                     |                               |           |
|---------------------|-------------------------------|-----------|
| <b>Приложение В</b> | <b>Дополнительные таблицы</b> | <b>90</b> |
|---------------------|-------------------------------|-----------|

# Введение

Обычная производная имеет целый порядок. Нулевая производная совпадает с самой функцией, первая показывает скорость изменения, вторая описывает изменение этой скорости. Дробное дифференцирование снимает ограничение на целый порядок и позволяет рассматривать производную порядка 0,5, 1,25 или 1,5. С вычислительной точки зрения меняется сама формула: вместо целого числа в порядок оператора подставляется дробное.

Здесь дробная производная взята прежде всего как вычислительная задача. Сама схема несложная. Выбираем формулу и пишем под неё программу. Дальше она считает значения для нескольких функций (берём  $x^2$ ,  $\sin x$ ,  $\cos x$ ,  $e^x$ ), а итог сводится в таблицы и графики. Подкупает то, что его каждый раз можно проверить числом. При  $\alpha = 1$  схема обязана давать почти обычную первую производную, и это сравнение делается буквально в одну строчку.

В качестве метода используется определение Грюнвальда–Летникова. Его преимущество для данной задачи в прямом переходе от формулы к алгоритму. Берутся значения функции в точках  $x$ ,  $x - h$ ,  $x - 2h$  и так далее, умножаются на коэффициенты и складываются, после чего сумма делится на  $h^\alpha$ . Интеграл с особенностью считать не нужно, и весь расчёт сводится к одному циклу.

Актуальность работы связана с тем, что в учебных курсах производная почти всегда вводится для целого порядка. При этом дробное дифференцирование не сводится к формальной записи. Операторы дробного порядка появляются там, где у системы есть память: так описывают вязкоупругие среды и аномальную диффузию, на их языке записаны и дробные дифференциальные уравнения. В учебной постановке дробный порядок проще понять через таблицы и графики. Особенно это видно на  $\sin x$  и  $\cos x$ , где при плавном изменении  $\alpha$  график постепенно переходит от исходной функции к её первой и второй производной.

**Цель работы** состоит в том, чтобы методом Грюнвальда–Летникова построить таблицу дробных производных элементарных функций и обосновать доверие к её значениям. Отсюда и задачи: разработка программного инструмента расчёта, сведение результатов в таблицы и графики для раз-

ных  $\alpha$  и оценка точности.

Цель распадается на несколько задач, и каждая из них привязана к конкретной части программы.

1. разобрать формулу Грюнвальда–Летникова и выделить параметры, от которых зависит расчёт;
2. коэффициенты  $c_k$  дробного порядка  $\alpha$  должны считаться по рекуррентной формуле, без факториалов;
3. нужна функция, вычисляющая дробную производную как в одной точке, так и сразу на сетке;
4. для набора элементарных функций значения дробной производной сводятся в общую таблицу по порядкам  $\alpha = 0, 0,5, 1, 1,5, 2$ , а для  $\sin x$  дополнительно строится подробная сетка с шагом 0,25;
5. проверка при  $\alpha = 1$  и  $\alpha = 2$  сопоставляет численный результат с известными производными;
6. зависимость от шага  $h$  и числа членов  $N$  исследуется отдельно, вместе с оценкой порядка точности;
7. наконец, собираются ограничения метода: краевой эффект, область определения, рост ошибки для быстрорастущих функций.

**Объектом исследования** являются дробные производные элементарных функций. **Предметом исследования** является численное вычисление этих производных по формуле Грюнвальда–Летникова и зависимость результата от параметров метода.

**Практическим результатом** стал небольшой программный комплекс: расчётные модули на Python и отдельный сайт на React. Python-часть строит таблицы, графики и гоняет вычислительные эксперименты. Сайт принимает функцию, порядок  $\alpha$ , шаг  $h$ , число членов  $N$  и границы интервала, считает прямо в браузере, рисует интерактивный график Plotly и выгружает таблицу в CSV. Это сознательно учебный инструмент, не больше. Таблицы и графики собраны вплотную к кодовым проверкам, чтобы сразу было видно, где расхождение идёт от самой формулы, а где

от выбранной длины истории. Для прикладных задач такой комплекс пришлось бы ещё отдельно проверять на устойчивость и точность.

Метод Грюнвальда–Летникова даёт численное приближение. Результат зависит от шага сетки, числа членов суммы и поведения функции на выбранном участке, и эту зависимость в работе разбирают через таблицы, графики и ошибки. Отдельно рассматривается, при каком шаге результат грубее, где растёт влияние округления и почему для дробного порядка нужна длинная история значений функции.

**Методы исследования** включают вычислительный эксперимент в среде Python, прямую реализацию разностной формулы, сопоставление численных значений с аналитическими решениями для целых и дробных порядков и анализ поведения погрешности при изменении параметров  $h$  и  $N$ .

**Структура работы.** Работа состоит из введения, пяти глав, заключения, списка литературы и приложений. В первой главе изложены теоретические основы дробного дифференцирования и выбор определения, во второй метод представлен как численный алгоритм, в третьей описана программная реализация, в четвёртой приведена сводная таблица дробных производных и вычислительные эксперименты вокруг неё, в пятой разобраны точность, устойчивость и пути её повышения.

# 1 Теоретические основы дробного дифференцирования

## 1.1 От обычной производной к производной дробного порядка

Производная первого порядка вводится через предел отношения приращений:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}. \quad (1)$$

Повторение операции  $n$  раз даёт производную целого порядка  $f^{(n)}(x) = \frac{d^n f}{dx^n}$ . Для целых порядков запись привычна:

$$D^0 f(x) = f(x), \quad D^1 f(x) = f'(x), \quad D^2 f(x) = f''(x). \quad (2)$$

Дробное дифференцирование ставит вопрос, как определить оператор  $D^\alpha$  при нецелом  $\alpha$ . Интуитивно  $D^{0,5} f(x)$  занимает промежуточное положение между функцией и её первой производной, а  $D^{1,5} f(x)$  располагается между первой и второй. В этой работе важна вычислительная постановка. Нужно посчитать значение для выбранной функции в выбранной точке, а затем проверить результат численно.

## 1.2 Почему дробную производную нельзя записать короткой разностью

Первую производную приближает левая разность с двумя значениями:

$$f'(x) \approx \frac{f(x) - f(x - h)}{h}. \quad (3)$$

Для второй производной нужно три значения, с коэффициентами 1, -2, 1:

$$f''(x) \approx \frac{f(x) - 2f(x - h) + f(x - 2h)}{h^2}. \quad (4)$$

Для дробного порядка такой короткой формулы недостаточно. В определении Грюнвальда–Летникова значение в точке  $x$  зависит от набора точек слева  $f(x)$ ,  $f(x - h)$ ,  $f(x - 2h)$ ,  $f(x - 3h)$ , ... Обычная производная в численном виде требует двух-трёх соседних точек, дробная использует длинную историю, и для неё в расчёт отдельно вводится параметр  $N$ , число предыдущих точек. При  $N = 500$  и шаге  $h$  программа в каждой точке берёт 501 значение функции, от  $f(x)$  до  $f(x - 500h)$ .

## 1.3 Левый вариант производной

Важно направление, с которого берутся значения. В работе выбрана левая производная. При вычислении в точке  $x$  используются точки  $x$ ,  $x - h$ , ...,  $x - Nh$  слева. Правый вариант с точками  $x + h$ ,  $x + 2h$ , ... тоже



возможен. Он получается из той же суммы с теми же коэффициентами заменой  $x - kh$  на  $x + kh$ .

Для дробного порядка левая и правая производные оказываются разными величинами. На  $\sin x$  они дают сдвиг фазы в противоположные стороны (рисунок 12). Левая схема для этой работы предпочтительнее, потому что её проще трактовать как расчёт по предыдущим значениям и проще контролировать у левой границы интервала. Выбор фиксируется явно, поскольку для дробной производной результат зависит и от определения, и от границы интервала.

## 1.4 Подходы к дробной производной и выбор схемы

У дробной производной нет единственного определения. Производная Римана–Лиувилля строится через дробный интеграл и обычное дифференцирование. При  $n - 1 < \alpha < n$

$$D_{RL}^{\alpha} f(x) = \frac{d^n}{dx^n} \left( \frac{1}{\Gamma(n - \alpha)} \int_a^x \frac{f(t)}{(x - t)^{\alpha - n + 1}} dt \right). \quad (5)$$

Здесь  $\Gamma$  обозначает гамма-функцию [1, 9]. Формула удобна для теории, но для первой программной реализации тяжелее, поскольку требуется численно считать интеграл с особенностью в ядре. Производная Капуто отличается порядком операций. Сначала берётся обычная производная, затем дробное интегрирование:

$$D_C^{\alpha} f(x) = \frac{1}{\Gamma(n - \alpha)} \int_a^x \frac{f^{(n)}(t)}{(x - t)^{\alpha - n + 1}} dt. \quad (6)$$

Она естественна в дифференциальных уравнениях, так как лучше согласуется с обычными начальными условиями [2, 9], но к теме разностных таблиц прямого отношения не имеет. Определение Грюнвальда–Летникова ближе всего к конечным разностям и сразу задаёт вычислительную схему.

Все три определения корректны, просто закрывают разные задачи. Риман–Лиувилль и Капуто сильнее в теории и в уравнениях с начальными условиями, зато требуют считать интеграл с особенностью в ядре. У Грюнвальда–Летникова интеграла нет, весь расчёт сводится к сумме по сетке, и для разностных таблиц это перевешивает остальное. Поэто-

му дальше используется именно оно. Для достаточно гладких функций (с непрерывными производными до порядка  $\lceil \alpha \rceil$  на отрезке  $[a, x]$ ) определения Грюнвальда–Летникова и Римана–Лиувилля совпадают [1, 9]; поэтому численные значения по Грюнвальду–Летникову далее сверяются с замкнутыми формами, выведенными для Римана–Лиувилля.

## 1.5 Определение Грюнвальда–Летникова

Левая дробная производная порядка  $\alpha$  записывается как предел [9, 1, 8]:

$$D_{GL}^\alpha f(x) = \lim_{h \rightarrow 0} \frac{1}{h^\alpha} \sum_{k=0}^{\lfloor (x-a)/h \rfloor} (-1)^k \binom{\alpha}{k} f(x - kh), \quad (7)$$

где  $a$  — левая граница,  $h$  — шаг,  $\alpha$  — порядок. Обобщённый биномиальный коэффициент равен

$$\binom{\alpha}{k} = \frac{\alpha(\alpha-1) \cdots (\alpha-k+1)}{k!}, \quad k \geq 1, \quad \binom{\alpha}{0} = 1. \quad (8)$$

Верхний параметр  $\alpha$  в обобщённой биномиальной формуле может быть дробным, и из-за этого коэффициенты ведут себя не так, как в целочисленной разности. При  $\alpha = 1$  остаются два коэффициента, отличных от нуля. При  $\alpha = 0,5$  последовательность продолжается дальше. Для расчёта предел заменяется конечной суммой:

$$D_{h,N}^\alpha f(x) = \frac{1}{h^\alpha} \sum_{k=0}^N c_k^{(\alpha)} f(x - kh), \quad c_k^{(\alpha)} = (-1)^k \binom{\alpha}{k}. \quad (9)$$

Формула (9) лежит в основе программы. В ней собраны все параметры расчёта: функция  $f$ , точка  $x$ , порядок  $\alpha$ , шаг  $h$  и число членов  $N$ .

## 1.6 Смысл коэффициентов

Коэффициент  $c_k^{(\alpha)}$  показывает, с каким весом значение  $f(x - kh)$  входит в сумму. Первый вес всегда равен единице,  $c_0^{(\alpha)} = 1$ , остальные зависят от  $\alpha$ . При  $\alpha = 1$  получается  $c_0 = 1$ ,  $c_1 = -1$ , остальные нули, и формула превращается в левую разность  $\frac{f(x) - f(x-h)}{h}$ . При  $\alpha = 2$  веса

равны  $1, -2, 1$ , как в разностной формуле второй производной. При дробном  $\alpha$  веса продолжают дальше. Например, при  $\alpha = 0,5$  они равны  $1, -0,5, -0,125, -0,0625, \dots$  и медленно затухают. Первые коэффициенты собраны в таблице 1. Они посчитаны программой.

Таблица 1 – Первые коэффициенты  $c_k^{(\alpha)}$  для нескольких порядков

| $\alpha$ | $c_0$ | $c_1$ | $c_2$  | $c_3$   | $c_4$   |
|----------|-------|-------|--------|---------|---------|
| 0        | 1     | 0     | 0      | 0       | 0       |
| 0,5      | 1     | -0,5  | -0,125 | -0,0625 | -0,0391 |
| 1        | 1     | -1    | 0      | 0       | 0       |
| 1,5      | 1     | -1,5  | 0,375  | 0,0625  | 0,0234  |
| 2        | 1     | -2    | 1      | 0       | 0       |

По этим числам делается первая сверка. Перепутанный знак коэффициента сразу ломает целые порядки  $\alpha = 1$  и  $\alpha = 2$ , и результат перестаёт походить на обычную производную.

## 1.7 Память метода и длина истории

Результат в точке  $x$  зависит от значений функции на участке  $[x - Nh, x]$ . Длину этого участка называют длиной истории:

$$L = Nh. \quad (10)$$

При  $h = 0,01$  и  $N = 500$  получается  $L = 5$ , то есть используются значения на отрезке длиной 5 слева от точки, и  $h$  с  $N$  выбирают с оглядкой на их произведение. Пара  $h = 0,01, N = 500$  и пара  $h = 0,005, N = 1000$  дают одинаковую историю  $L = 5$ , но разную плотность сетки. Влияние длины истории показано в разделе 4.6.

Память дробной производной связана со скоростью затухания коэффициентов. При больших  $k$  величина  $|c_k^{(\alpha)}|$  убывает примерно как  $k^{-(\alpha+1)}$  [9]. Такое степенное затухание медленное. Для целого порядка коэффициенты обрываются точно, поэтому обычная производная локальна (график в разделе 4.7).

## 1.8 Эталонные формулы для проверки

Часть результатов сравнивается с аналитическими формулами [8, 1, 9]. Для степенной функции при  $x > 0$  и нижнем пределе  $a = 0$

$$D^\alpha x^\beta = \frac{\Gamma(\beta + 1)}{\Gamma(\beta - \alpha + 1)} x^{\beta - \alpha}. \quad (11)$$

При целых порядках это согласуется с обычным дифференцированием: при  $\beta = 2, \alpha = 1$  получаем  $2x$ , при  $\alpha = 2$  выходит  $2$ . Для  $\sin x$  и  $\cos x$  в форме Лиувилля (так называют вариант определения с нижним пределом  $a \rightarrow -\infty$ , когда история уходит неограниченно влево) удобен сдвиг фазы:

$$D^\alpha \sin x = \sin\left(x + \frac{\pi\alpha}{2}\right), \quad D^\alpha \cos x = \cos\left(x + \frac{\pi\alpha}{2}\right). \quad (12)$$

Для  $e^x$  в той же форме  $D^\alpha e^x = e^x$ . Дальше эти формулы используются для сверки. Численная сумма на конечной истории зависит от  $N$  и левой границы, и полного совпадения, особенно для  $\sin x$  и  $e^x$ , ожидать не следует.

Собранные в одном месте, эти замкнутые формы и образуют ту самую справочную таблицу дробных производных элементарных функций, что вынесена в название работы (таблица 2). Численная схема Грюнвальда–Летникова в следующих главах воспроизводит её строку за строкой и показывает, насколько близко к этим формулам подходит расчёт. Для  $\ln x$  и  $\tan x$  замкнутой формулы в элементарных функциях нет (она выражается через специальные функции), а у комбинированных функций она получается через обобщённое правило Лейбница и слишком громоздка, чтобы держать её в таблице; такие функции остаются только в численной части. Численный аналог этой справочной таблицы, собранные в одном месте значения  $D^\alpha f$ , приведён в разделе 4.1 (таблица 7).

## 1.9 Что из теории используется дальше

В программу из первой главы переходит несколько готовых решений. Выбрана левая производная, значит программа берёт точки  $x - kh$ . Порядок  $\alpha$  остаётся параметром, и код не пишется только под  $\alpha = 0,5$  или только под целые порядки. Коэффициенты зависят от  $\alpha$  и пересчитываются при его изменении. Результат зависит от длины истории  $Nh$ , поэтому в

Таблица 2 – Сводная таблица дробных производных элементарных функций

| Функция $f(x)$    | Дробная производная $D^\alpha f(x)$                                                   | Режим / условие       |
|-------------------|---------------------------------------------------------------------------------------|-----------------------|
| $x^\beta$         | $\frac{\Gamma(\beta + 1)}{\Gamma(\beta - \alpha + 1)} x^{\beta - \alpha}$             | $a = 0, x > 0$        |
| 1                 | $\frac{x^{-\alpha}}{\Gamma(1 - \alpha)}$                                              | $a = 0$ ; по Капуто 0 |
| $e^x$             | $e^x$                                                                                 | форма Лиувилля        |
| $\sin x$          | $\sin\left(x + \frac{\pi\alpha}{2}\right)$                                            | форма Лиувилля        |
| $\cos x$          | $\cos\left(x + \frac{\pi\alpha}{2}\right)$                                            | форма Лиувилля        |
| $\sin x + \cos x$ | $\sin\left(x + \frac{\pi\alpha}{2}\right) + \cos\left(x + \frac{\pi\alpha}{2}\right)$ | форма Лиувилля        |
| $\ln x, \tan x$   | нет элементарной замкнутой формы                                                      | только численно       |
| $x \sin x, xe^x$  | замкнутая форма громоздка (правило Лейбница)                                          | только численно       |

экспериментах отдельно рассматривается влияние  $h$ ,  $N$  и их произведения.

## 2 Метод Грюнвальда–Летникова как численный алгоритм

### 2.1 От определения к расчёту

Предел в определении заменяется конечной разностной суммой (9). Эта сумма уже является алгоритмом. Чтобы получить одно значение, нужно пройти по  $k$  от 0 до  $N$ , посчитать аргумент  $x - kh$ , взять значение функции, умножить на коэффициент и прибавить к сумме. Если строится таблица на сетке  $x_0, \dots, x_m$ , расчёт повторяется для каждой точки, и время работы зависит и от числа точек таблицы, и от числа членов суммы.

### 2.2 Точная производная и численная схема

Теоретический оператор и его численное приближение различаются. В роли теоретического объекта выступает левая производная  $D_{GL}^\alpha f(x)$  с нижней границей  $a$ . В программе используется конечная сумма  $D_{h,N}^\alpha f(x)$  с фиксированными  $h$  и  $N$ . При  $h \rightarrow 0$  и  $N = \lfloor (x - a)/h \rfloor$  сумма стремится к оператору с нижним пределом  $a$ . При фиксированной длине истории  $L = Nh$  получается приближение с конечной памятью. Поэтому в работе исследуется аппроксимация при выбранных  $h$  и  $N$ , а точные формулы привлекаются для контроля.

### 2.3 Режимы вычисления: нижний предел и длина истории

Схема  $D_{h,N}^\alpha$  зависит от того, докуда тянется история, и в работе различаются три таких режима (таблица 3). Для одной и той же функции они дают разные числа, поскольку каждый режим задаёт свой нижний предел и свою длину памяти. Эти различия относятся к постановке расчёта и не считаются ошибкой реализации.

Каждая таблица и каждый график подписаны режимом, в котором

Таблица 3 – Режимы вычисления дробной производной

| Режим                   | Параметры                   | С чем сравнивается                                         |
|-------------------------|-----------------------------|------------------------------------------------------------|
| Нижний предел $a = 0$ . | $N = \lfloor x/h \rfloor$ . | Степенная формула (11) через $\Gamma$ .                    |
| Конечная память.        | $L = Nh$ фиксировано.       | Приближение по короткой истории, без собственного эталона. |
| Приближение Лиувилля.   | $L = Nh$ большое.           | Фазовые формулы (12) для $\sin x$ , $\cos x$ , $e^x$ .     |

посчитаны. Степенные функции  $x^2$ ,  $x^3$  считаются в режиме  $a = 0$ , поэтому сравниваются с гамма-формулой. Число членов в этом режиме берётся как  $N = \lfloor x/h \rfloor$ , и при  $x$ , не кратном  $h$ , самый левый узел  $x - Nh$  попадает не ровно в нуль, а в полуинтервал  $[0, h)$ ; это смещение само имеет порядок  $h$ , укладывается в общую погрешность  $O(h)$  и далее отдельно не выделяется. Тригонометрические функции и  $e^x$  считаются с фиксированной длиной истории  $L = Nh = 5$ , и фазовые формулы дают для них лишь приближённую картину. Численная иллюстрация различия между режимами приведена в разделе 5.6.

## 2.4 Расчёт коэффициентов без факториалов

Прямой расчёт через произведение и факториал при больших  $k$  требует лишних операций и больших промежуточных чисел. Из свойства  $\binom{\alpha}{k} = \binom{\alpha}{k-1} \frac{\alpha - k + 1}{k}$  и определения  $c_k = (-1)^k \binom{\alpha}{k}$  получается рекуррентное правило:

$$c_0^{(\alpha)} = 1, \quad c_k^{(\alpha)} = c_{k-1}^{(\alpha)} \cdot \frac{k - 1 - \alpha}{k}, \quad k = 1, 2, \dots, N. \quad (13)$$

На каждый коэффициент уходит одно умножение и одно деление. Массив длиной  $N + 1$  создаётся один раз и используется для всех точек графика при фиксированных  $\alpha$  и  $N$ .

## 2.5 Схема вычисления одного значения

Сначала выбираются параметры  $\alpha, h, N$  и считаются коэффициенты  $c_0, \dots, c_N$ . Затем для точки  $x$  строятся аргументы  $x_0 = x, x_1 = x - h, \dots, x_N = x - Nh$ , значения функции складываются с весами

$$S = c_0 f(x_0) + c_1 f(x_1) + \dots + c_N f(x_N), \quad (14)$$

и итог получается делением на  $h^\alpha$ :  $D_{h,N}^\alpha f(x) = S/h^\alpha$ . В схеме несколько мест, где легко ошибиться. Это знак коэффициентов, степень  $h^\alpha$ , выход за область определения и порядок обхода точек. Все они проверяются на целых порядках.

## 2.6 Пример ручного расчёта

Разберём небольшой пример. Возьмём  $f(x) = x^2$ , порядок  $\alpha = 0,5$ , шаг  $h = 0,1$ , число членов  $N = 3$ , точку  $x = 1$ . Коэффициенты при  $\alpha = 0,5$  равны  $c_0 = 1, c_1 = -0,5, c_2 = -0,125, c_3 = -0,0625$ . Точки сетки: 1, 0,9, 0,8, 0,7; значения функции: 1, 0,81, 0,64, 0,49. Сумма до деления:

$$S = 1 \cdot 1 - 0,5 \cdot 0,81 - 0,125 \cdot 0,64 - 0,0625 \cdot 0,49 = 0,484375. \quad (15)$$

После деления на  $h^{0,5} = \sqrt{0,1}$  получается  $D^{0,5}x^2|_{x=1} \approx 0,484375/0,3162 \approx 1,532$ . Пример посчитан с маленьким  $N$ , и история здесь короткая. Самый левый узел стоит не в нуле, а в точке  $x - Nh = 0,7$ , то есть перед нами конечная память  $L = 0,3$ , а не режим  $a = 0$ . Поэтому эталон  $\Gamma(3)/\Gamma(2,5) \approx 1,505$  из формулы (11), выведенной как раз для  $a = 0$ , служит здесь лишь ориентиром, и совпадение приблизительное. Зато на нём хорошо виден сам ход расчёта, сводящийся к обычной арифметике. При больших  $N$  и мелком шаге результат уже приближается к эталону (раздел 4.8).

## 2.7 Нормировка на $h^\alpha$

Делитель  $h^\alpha$  задаёт масштаб производной нужного порядка. При  $\alpha = 1$  это деление на  $h$ , при  $\alpha = 2$  на  $h^2$ , при  $\alpha = 0,5$  на  $\sqrt{h}$ . На этом множителе легко ошибиться. Если деление пропустить, форма графика сохранится,



масштаб исказится, и расхождение проявится при сравнении результатов для разных  $h$ .

## 2.8 Параметры $\alpha$ , $h$ , $N$

Порядок  $\alpha$  задаёт операцию. В работе используются значения  $\alpha \in \{0, 0,25, 0,5, 0,75, 1, 1,25, 1,5, 1,75, 2\}$ . Шаг  $h$  задаёт расстояние между соседними точками суммы. Число  $N$  задаёт количество слагаемых и длину истории  $L = Nh$ . Важно, что одну и ту же историю можно набрать разной сеткой (таблица 4).

Таблица 4 – Связь параметров  $h$ ,  $N$ ,  $L = Nh$

| $h$   | $N$  | $L = Nh$ |
|-------|------|----------|
| 0,1   | 50   | 5        |
| 0,01  | 500  | 5        |
| 0,005 | 1000 | 5        |
| 0,01  | 100  | 1        |

Первые три строки дают одну и ту же историю  $L = 5$  при всё более плотной сетке, и за эту плотность приходится платить временем счёта. Последняя строка показывает обратный случай. При коротком  $N$  история проседает до  $L = 1$ , и для дробного порядка результат заметно портится. За базовую точку экспериментов взято  $h = 0,01$ ,  $N = 500$ , история  $L = 5$ . Подбиралось это не как строгий оптимум, а скорее на глаз: графики уже достаточно гладкие, дробная память не обрывается сразу, а пересчёт всех таблиц и рисунков остаётся быстрым. От этой точки удобно шагать в обе стороны и смотреть, что меняется при более грубом шаге или более короткой истории.

## 2.9 Краевой эффект и область определения

Так как используются точки  $x - kh$ , у левой границы интервала часть точек может уйти левее допустимой области. На интервале  $[0, 10]$  при  $h = 0,01$ ,  $N = 500$  для точки  $x = 1$  самая левая точка равна  $1 - 500 \cdot 0,01 = -4$ . Для функций, определённых на всей прямой ( $\sin$ ,  $\cos$ ,  $e^x$ ), такой выход

допустим. Для  $\ln x$  нужно соблюдать условие  $x - Nh > 0$ , иначе в сумме появится логарифм неположительного числа. При  $h = 0,01, N = 500$  это даёт  $x > 5$ , поэтому график строится на  $[6, 10]$ . В программе функция  $\ln$  возвращает NaN вне области определения, и результат в таких точках тоже становится NaN, что позволяет явно отметить некорректные точки.

## 2.10 Трудоемкость и источники ошибки

Если таблица строится в  $m$  точках и в каждой используется  $N + 1$  слагаемое, число операций пропорционально  $m(N + 1)$ , то есть  $O(mN)$ . Векторизованная реализация строит матрицу узлов размера  $m \times (N + 1)$ , поэтому потребление памяти также имеет порядок  $O(mN)$ . Для больших  $N$  или больших сеток это ограничение существенно, и расчёт пришлось бы вести блоками. При  $m = 200, N = 500$  нужно около 100 000 слагаемых, что для Python приемлемо.

Ошибка приближения складывается из нескольких независимых источников, и каждый из них разбирается в работе отдельно. Шаг  $h$  задаёт грубость сетки, а при очень малых значениях начинает усиливать округление; его влияние прослеживается на ряде  $h = 0,1 \dots 0,001$  (раздел 4.5). Число  $N$  обрывает сумму и укорачивает память, поэтому диапазон  $N = 5 \dots 1000$  вынесен в раздел 4.6. У левой границы точки  $x - kh$  могут уйти за область определения, и расчёт держат там, где  $x - Nh$  ещё попадает в неё. Конечная точность вещественных чисел добавляет порог округления, заметный при малом  $h$  (раздел 5.4).

## 2.11 Способы оценки ошибки

При целых порядках численный результат сравнивается с классической производной  $g(x)$ . Абсолютная ошибка в точке равна  $\varepsilon_i = |D_{h,N}^\alpha f(x_i) - g(x_i)|$ , относительная равна  $\varepsilon_i / \max(1, |g(x_i)|)$ . Знаменатель здесь намеренно ограничен снизу единицей: иначе у нулей  $\sin x$  и  $\cos x$ , где  $g(x_i) \rightarrow 0$ , относительная ошибка обращалась бы в бесконечность. У такой обрезки есть обратная сторона. Там, где  $|g(x_i)| < 1$ , деление идёт на единицу, и приведённая величина совпадает с абсолютной ошибкой, занижая истинную относительную. Поэтому смысл настоящей относительной ошибки она

несёт лишь при  $|g(x_i)| \geq 1$ , как для  $e^x$  при росте  $x$ ; у тригонометрических функций возле нулей её следует читать как абсолютную. Для всей таблицы считаются максимальная и средняя ошибки:

$$\varepsilon_{\max} = \max_i \varepsilon_i, \quad \varepsilon_{\text{avg}} = \frac{1}{m} \sum_{i=1}^m \varepsilon_i. \quad (16)$$

Максимальная ошибка чувствительна к худшей точке (часто у края), средняя отражает поведение на всём интервале.

## 3 Программная реализация

### 3.1 Выбор инструментов

Основная вычислительная часть написана на Python [11]. Для численного эксперимента он оправдан. Математические функции можно передавать в расчёт как объекты, `numpy` быстро работает с массивами, а таблицы и графики строятся стандартными библиотеками.

Роли библиотек разделены по задачам. `numpy` отвечает за массивы и вычисления [4, 10]. `pandas` используется для таблиц и экспорта в CSV и Excel, `matplotlib` строит графики [5], `scipy` считает гамма-функцию в эталонных формулах. Интерактивная часть вынесена в отдельный веб-проект на React и Vite [12, 13]. График в браузере строится через Plotly. На нём можно наводить курсор на кривые, смотреть значения, масштабировать участок и возвращаться к исходному виду [14]. Браузерная версия повторяет ту же формулу на стороне клиента, поэтому инструмент работает без Python-бэкенда; её вывод сверялся с расчётной частью на наборе контрольных точек (отдельные тесты `bun test` воспроизводят те же эталонные значения, что и Python). Веб-проект собирается командой `bun run build` [15].

Сумма по сетке записана как умножение матрицы узлов на вектор коэффициентов. Благодаря этому основная нагрузка переносится на оптимизированные операции `numpy`, а явные циклы по точкам таблицы из кода убираются. Это относится к основному режиму с фиксированным числом членов  $N$ . В режиме нижнего предела  $a = 0$  длина истории  $N = \lfloor x/h \rfloor$

своя у каждой точки, поэтому там сумма по сетке считается поточечно и цикл по узлам остаётся.

## 3.2 Структура проекта

Код разбит на модули, чтобы формула метода, список функций и эксперименты не смешивались (таблица 5).

Таблица 5 – Структура программного проекта

| Файл                        | Назначение                                                                                                 |
|-----------------------------|------------------------------------------------------------------------------------------------------------|
| <code>fractional.py</code>  | Коэффициенты и дробная производная в точке и на сетке.                                                     |
| <code>functions.py</code>   | Набор функций с эталонами и областью определения.                                                          |
| <code>experiments.py</code> | Расчётные эксперименты, возвращают таблицы чисел.                                                          |
| <code>tables.py</code>      | Сохранение таблиц в CSV и Excel.                                                                           |
| <code>plots.py</code>       | Построение и сохранение графиков.                                                                          |
| <code>main.py</code>        | Точка входа: расчёт из командной строки и сборка всего.                                                    |
| <code>web/</code>           | Веб-интерфейс на React, Vite и Plotly. Настройка параметров, интерактивный график, таблица и выгрузка CSV. |
| <code>tests/</code>         | Автоматические тесты реализации (pytest).                                                                  |

## 3.3 Коэффициенты и производная

В основе программы лежат два коротких фрагмента. Первый считает коэффициенты по рекуррентной формуле (13). Та же рекуррента есть накопленное произведение отношений  $\frac{k-1-\alpha}{k}$ , поэтому весь массив получается одним вызовом `cumprod` без явного цикла по  $k$ :

```
def gl_coefficients(alpha, n):
    c = np.empty(n + 1)
    c[0] = 1.0
```

```

if n >= 1:
    k = np.arange(1, n + 1)
    c[1:] = np.cumprod((k - 1 - alpha) / k)
return c

```

Второй считает производную сразу на сетке точек. Коэффициенты считаются один раз, а матрица узлов `nodes` содержит для каждой точки её историю длиной  $N$ :

```

def gl_grid(f, xs, alpha, h, n):
    coeffs = gl_coefficients(alpha, n)
    xs = np.asarray(xs, dtype=float)
    k = np.arange(n + 1)
    nodes = xs[:, None] - k[None, :] * h
    vals = np.asarray(f(nodes), dtype=float)
    return vals @ coeffs / h ** alpha

```

Здесь видна прямая связь теории и кода. Порядок  $\alpha$  входит в коэффициенты и в делитель  $h^\alpha$ , шаг  $h$  задаёт сетку `nodes`, число  $N$  ограничивает длину истории. Если функция не определена в какой-то точке истории, она возвращает NaN, и результат в этой точке тоже становится NaN.

## 3.4 Формирование таблиц и графиков

Модуль `tables.py` собирает значения дробной производной для набора порядков и сохраняет их в CSV и в файл `tables.xlsx` (по листу на таблицу). Модуль `plots.py` строит графики и сохраняет их в PNG. Все числа берутся из одного модуля `experiments.py`, так что таблицы и графики согласованы между собой.

## 3.5 Запуск

Расчёт в одной точке или на сетке запускается из командной строки:

```

python main.py --function sin --alpha 0.5 --h 0.01 --n 500 --x 1.5708
python main.py --function x2 --alpha 0.5 --h 0.01 --n 500 --xmin 1 --xmax 5 --points 9 --
    csv out.csv

```

Сборка всех таблиц и графиков работы выполняется командой `python main.py --all`. Зависимости Python перечислены в `requirements.txt` и ставятся командой `pip install -r requirements.txt`.

Веб-интерфейс запускается отдельно:

```
cd web
bun install
bun run dev
```

Для подготовки статической версии используется команда `bun run build`.

## 3.6 Веб-интерфейс

Интерактивная часть вынесена в каталог `web/`. Это отдельный проект на React, TypeScript, Vite и Plotly. Локальный запуск выполняется командами `bun install` и `bun run dev`. Для публикации сайта используется сборка `bun run build`. Результат появляется в каталоге `web/dist/` и может быть размещён на статическом хостинге. Рабочая версия интерфейса опубликована по адресу <https://course-work-2026.vercel.app/>. Исходный код, текст работы и материалы проекта размещены в репозитории <https://github.com/SaltyFrappuccino/CourseWork-2026>.

В интерфейсе выбирается функция из списка, задаётся порядок  $\alpha \in [0, 2]$ , шаг  $h$ , число членов  $N$  и границы интервала. Доступны те же схемы, что и в расчётной части: стандартная Грюнвальда–Летникова с фиксированным  $N$ , режим нижнего предела  $a = 0$  (число членов задаётся как  $N = \lfloor x/h \rfloor$ , и для степенных функций значения совпадают с формулой (11) через  $\Gamma$ ), сдвинутый вариант ( $p = 1$ ) и Капуто. В режимах  $a = 0$  и Капуто поле  $N$  становится неактивным, поскольку число членов определяется самой точкой. При выборе схемы Капуто диапазон порядка автоматически сужается до  $[0,05; 0,95]$  внутри интервала  $(0, 1)$ , на котором схема L1 определена (значения у самых краёв,  $\alpha \rightarrow 0$  и  $\alpha \rightarrow 1$ , исключены, так как там формула вырождается), и уже выставленное значение  $\alpha$  при необходимости подтягивается внутрь этого диапазона. Центральная часть отведена под интерактивный график исходной функции и дробной производной. Курсор показывает значения в выбранной точке, колесо мыши масштабирует участок, а панель графика двигает область просмотра и сбрасывает приближение. Ось  $Y$  подбирается автоматически под текущие значения, поскольку при изменении  $\alpha$  меняется и масштаб дробной производной. Справа выводится таблица значений. Для целых порядков  $\alpha = 0, 1, 2$  дополнительно показывается эталонная обычная производная и

численная оценка отклонения. Эта оценка считается в арифметике двойной точности и служит быстрой машинной проверкой реализации.

Здесь важна одна деталь реализации. Специальные функции в браузере написаны заново и не повторяют `scipy`. Гамма-функцию, нужную для масштабного множителя схемы L1 Капуто, веб-версия считает приближением Стирлинга. Фактически это вторая, независимая реализация метода. Её ядро (коэффициенты и сумма Грюнвальда–Летникова) сверяется с Python на контрольных точках тестами `bun test`, и совпадение значений подтверждает, что обе ветви считают одно и то же.

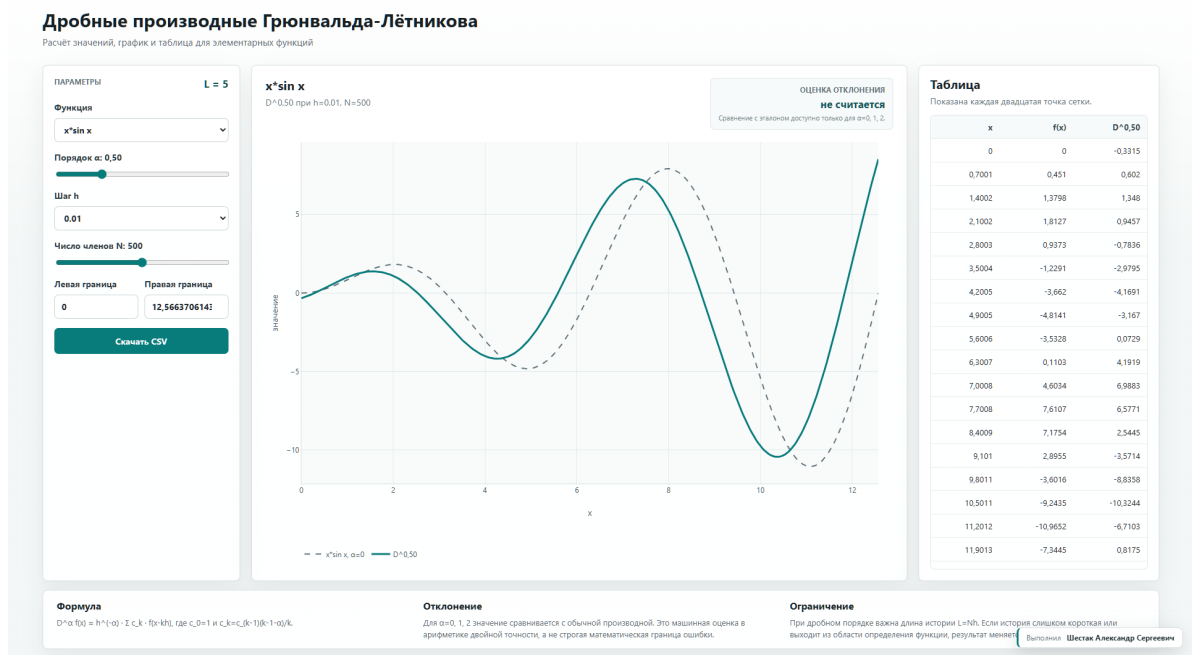


Рисунок 1 – Веб-интерфейс программы с параметрами метода, интерактивным графиком Plotly и таблицей значений

### 3.7 Тестирование программы

Корректность реализации контролируется на свойствах, которые не зависят от конкретной функции, и на сравнении с известными производными (таблица 6). Эти проверки оформлены как автоматические тесты в каталоге `tests/` и запускаются командой `pytest` из корня проекта. Влияние округления при малом  $h$  дополнительно рассматривается в разделе 5.4. Тесты не доказывают корректность полностью, но ловят грубые ошибки, включая неверный знак, забытое деление на  $h^{\alpha}$ , неправильный порядок точек и ошибку в рекуррентной формуле.

Таблица 6 – Проверки реализации

| Тест                           | Ожидаемый результат             | Что проверяет                                 |
|--------------------------------|---------------------------------|-----------------------------------------------|
| $\alpha = 0.$                  | $D^0 f = f.$                    | Базовую нормировку.                           |
| $\alpha = 1, \sin x.$          | Близко к $\cos x.$              | Знак и деление на $h.$                        |
| $\alpha = 2, x^2.$             | Ровно 2.                        | Коэффициенты второго порядка.                 |
| $\alpha = 0,5, x^2, a = 0.$    | Формула через $\Gamma.$         | Дробный порядок.                              |
| Сумма коэффициентов.           | Стремится к нулю.               | Рекуррентную формулу.                         |
| $\ln x$ при выходе за область. | NaN.                            | Обработку области определения.                |
| Малый шаг $h.$                 | Рост ошибки.                    | Влияние округления.                           |
| Ячейки сводной таблицы.        | Совпадают с $\Gamma$ -формулой. | Композицию схем в <code>master_table</code> . |



## 4 Вычислительные эксперименты

### 4.1 Сводная таблица дробных производных элементарных функций

Главный результат работы — собранная численным методом таблица дробных производных элементарных функций (таблица 7). По строкам в ней идут функции из набора `functions.py`, по столбцам — порядки  $\alpha = 0, 0,5, 1, 1,5, 2$ ; в ячейке стоит значение  $D^\alpha f$  в опорной точке  $x_0$ , посчитанное по формуле (9). Каждая функция взята в своём естественном режиме (раздел 2.3). Степенные считаются с нижним пределом  $a = 0$ , остальные с конечной памятью  $L = Nh = 5$  (для  $\tan x$  короче,  $L = 1$ ). Замкнутые формы для тех же функций собраны в справочной таблице 2; численная схема воспроизводит их строка за строкой, а вся остальная часть главы показывает, почему её числам можно верить.

Таблица 7 – Сводная таблица численных дробных производных  $D^\alpha f$  элементарных функций по Грюнвальду–Летникову ( $h = 0,01$ )

| $f(x)$            | $x_0$ | Режим          | $D^0$ | $D^{0,5}$ | $D^1$  | $D^{1,5}$ | $D^2$  |
|-------------------|-------|----------------|-------|-----------|--------|-----------|--------|
| 1                 | 2     | $a = 0$        | 1,000 | 0,399     | 0,000  | −0,100    | 0,000  |
| $x$               | 2     | $a = 0$        | 2,000 | 1,595     | 1,000  | 0,400     | 0,000  |
| $x^2$             | 2     | $a = 0$        | 4,000 | 4,247     | 3,990  | 3,186     | 2,000  |
| $x^3$             | 2     | $a = 0$        | 8,000 | 10,181    | 11,940 | 12,694    | 11,940 |
| $e^x$             | 1     | память $L = 5$ | 2,718 | 2,712     | 2,705  | 2,698     | 2,691  |
| $\sin x$          | 1     | память $L = 5$ | 0,841 | 0,997     | 0,545  | −0,212    | −0,836 |
| $\cos x$          | 1     | память $L = 5$ | 0,540 | −0,197    | −0,839 | −0,982    | −0,549 |
| $\sin x + \cos x$ | 1     | память $L = 5$ | 1,382 | 0,800     | −0,294 | −1,193    | −1,385 |
| $\ln x$           | 8     | память $L = 5$ | 2,079 | 0,705     | 0,125  | −0,023    | −0,016 |
| $x \sin x$        | 1     | память $L = 5$ | 0,841 | 1,011     | 1,381  | 1,287     | 0,270  |
| $xe^x$            | 1     | память $L = 5$ | 2,718 | 4,059     | 5,396  | 6,725     | 8,047  |
| $\tan x^*$        | 1     | память $L = 1$ | 1,557 | 2,287     | 3,373  | 5,418     | 10,125 |

\* Строка  $\tan x$  дана как иллюстрация роста вблизи разрыва  $\pi/2$ : замкнутого эталона у неё нет, и само число зависит от выбранной памяти.

Прежде чем разбирать числа, стоит объяснить, почему режим у строк разный. Сделано это намеренно. Единого нижнего предела, при котором у всех перечисленных функций сразу нашлась бы замкнутая элементарная форма для сверки, попросту не существует. При  $a = 0$  степенные функ-

ции дают чистую формулу через гамма-функцию (11), но у  $\sin x$ ,  $\cos x$  и  $e^x$  от того же предела появляется начальный переходный член, и простого фазового сдвига уже нет. В форме Лиувилля ( $a \rightarrow -\infty$ ), наоборот, фазовые формулы (12) для тригонометрии и экспоненты точны, зато степенная сумма по бесконечной истории расходится, ведь слагаемые  $(x - kh)^\beta$  растут быстрее, чем убывают коэффициенты. Поэтому каждая функция взята в единственном режиме, где у неё есть чистый аналитический эталон; режим выписан отдельным столбцом, а сверка каждой ячейки со своей замкнутой формой вынесена в таблицу 8. По сути таблица 7 собрана как численный атлас, где каждая строка взята в своём, естественном для функции режиме. Тот же оператор можно применить ко всем функциям и с общим пределом  $a = 0$ ; тогда тригонометрия и экспонента заметно отходят от фазовых формул. Такой единый расчёт приведён в приложении (таблица 24). Он показывает, что разные режимы в основной таблице выбраны ради удобной сверки с эталонами, а сам метод одинаково считает все функции.

Там, где эталонная формула есть, таблица с ней согласуется; полная сверка для двух дробных порядков собрана в таблице 8. Для степенных функций значения в режиме  $a = 0$  совпадают с формулой (11) через гамма-функцию. При  $\alpha = 0,5$  численные 4,247 ( $x^2$ ), 1,595 ( $x$ ) и 10,181 ( $x^3$ ) отвечают точным 4,255, 1,596 и 10,213, с отклонением порядка  $h$ . Производная постоянной при дробном  $\alpha$  нулю не равна:  $D^{0,5}1 = 0,399$ , что отвечает  $x^{-\alpha}/\Gamma(1 - \alpha)$  и сразу выдаёт память метода. Для  $\sin x$ ,  $\cos x$  и их суммы столбец  $\alpha = 1$  даёт почти точную первую производную ( $D^1 \sin x|_{x=1} = 0,545$  против  $\cos 1 = 0,540$ ), а дробные столбцы приближают фазовые формулы (12) с поправкой на конечную историю. Именно эта поправка и видна в таблице 8. У тригонометрии и  $e^x$  остаётся зазор порядка процента от амплитуды, тогда как степенные в режиме  $a = 0$  ложатся на эталон почти точно. Строки  $\ln x$ ,  $\tan x$ ,  $x \sin x$ ,  $xe^x$  замкнутой формулы не имеют и приводятся только численно; у  $\tan x$  значения быстро растут с  $\alpha$  из-за близости разрыва.

Дальше каждый столбец и каждая строка этой таблицы разбираются по отдельности: переход формы при росте  $\alpha$ , проверка целых порядков, влияние шага  $h$  и числа членов  $N$ , оценки точности и устойчивости. По сути вся глава обосновывает, что значениям таблицы 7 можно верить.

Таблица 8 – Сверка сводной таблицы 7 с замкнутыми формами (точки  $x_0$  те же). Эталон для степенных задаёт формула (11) через  $\Gamma$  в режиме  $a = 0$ , для остальных его роль играет форма Лиувилля (12); зазор у  $\sin x$ ,  $\cos x$  и  $e^x$  отражает конечную память  $L = 5$

| $f(x)$            | числ. $D^{0,5}$ | замкн. $D^{0,5}$ | числ. $D^{1,5}$ | замкн. $D^{1,5}$ |
|-------------------|-----------------|------------------|-----------------|------------------|
| 1                 | 0,399           | 0,399            | −0,100          | −0,100           |
| $x$               | 1,595           | 1,596            | 0,400           | 0,399            |
| $x^2$             | 4,247           | 4,255            | 3,186           | 3,192            |
| $x^3$             | 10,181          | 10,213           | 12,694          | 12,766           |
| $e^x$             | 2,712           | 2,718            | 2,698           | 2,718            |
| $\sin x$          | 0,997           | 0,977            | −0,212          | −0,213           |
| $\cos x$          | −0,197          | −0,213           | −0,982          | −0,977           |
| $\sin x + \cos x$ | 0,800           | 0,764            | −1,193          | −1,190           |

## 4.2 План экспериментов

В основной части рассматриваются функции  $x$ ,  $x^2$ ,  $\sin x$ ,  $\cos x$ ,  $e^x$ , а в качестве расширения берутся  $\ln x$ ,  $x^3$  и комбинированные функции  $x \sin x$ ,  $xe^x$ ,  $\sin x + \cos x$ . Все они зарегистрированы в `functions.py` и доступны как из командной строки, так и в интерфейсе. Их таблицы сохраняются в каталог `tables/` (например,  $x \sin x$  попадает в `xsin_values.csv`). Если не сказано иное, базовые параметры  $h = 0,01$ ,  $N = 500$ .

Набор функций подобран не случайно. Главное требование: аналитический эталон хотя бы в одном режиме. Степенные  $x$ ,  $x^2$ ,  $x^3$  дают строгую сверку через гамма-формулу (11); для  $\sin x$ ,  $\cos x$  и  $e^x$  работают фазовые формулы (12) в форме Лиувилля; целые порядки любой функции сверяются с классической производной. Дальше хотелось охватить разные типы поведения, чтобы на таблице были видны и согласие с эталоном, и границы метода. Сюда вошли ограниченные колебания  $\sin x$  и  $\cos x$  и, для контраста, быстро растущие  $e^x$  и  $x^3$ . Добавлены и два неудобных для метода случая: у  $\ln x$  область определения ограничена снизу, а  $\tan x$  и вовсе имеет разрыв. Простейшие же комбинации  $x \sin x$ ,  $xe^x$ ,  $\sin x + \cos x$  показывают, что схема работает и там, где замкнутой формулы уже нет, а функция остаётся элементарной. Что-то вроде  $\sqrt{x}$  или  $a^x$  к этому ничего не добавляет, поэтому в набор не вошло.

Сводка экспериментов приведена в таблице 9.

Таблица 9 – Параметры вычислительных экспериментов

| Эксперимент               | Функция, $\alpha$             | Эталон                    |
|---------------------------|-------------------------------|---------------------------|
| Переход формы по порядку. | $\sin x, \alpha = 0 \dots 2.$ | $\cos x, -\sin x.$        |
| Проверка целых порядков.  | $x^2, e^x, \sin x, \cos x.$   | Классическая производная. |
| Влияние шага $h$ .        | $\sin x, \alpha = 1.$         | $\cos x.$                 |
| Влияние числа $N$ .       | $\sin x, \alpha = 0,5.$       | $\sin(x + \pi/4).$        |
| Сходимость по $h$ .       | $\sin x$ и $x^2.$             | $\cos 1$ и формула Г.     |
| Степенные функции.        | $x^2, x^3$ , режим $a = 0.$   | Формула (11).             |

### 4.3 Изменение функции при росте порядка производной

Проще всего рост порядка читается на  $\sin x$ , где кривая плавно переходит от исходной функции к её производным (рисунок 2). При  $\alpha = 0$  это сам  $\sin x$ , при  $\alpha = 1$  кривая ложится на  $\cos x$ , при  $\alpha = 2$  на  $-\sin x$ . Промежуточные порядки  $\alpha = 0,5$  и  $\alpha = 1,5$  дают плавный сдвиг фазы. Каждая такая кривая получается как взвешенная сумма значений функции в предыдущих точках. На целых порядках она ложится на пунктирные эталоны  $\cos x$  и  $-\sin x$ .

Те же значения в табличном виде приведены в таблице 10. В строке  $x = \pi/2 \approx 1,571$  видно, как растёт порядок:  $f = 1$ , при  $\alpha = 0,5$  значение 0,732, при  $\alpha = 1$  значение 0,005, то есть  $\cos(\pi/2) = 0$  с точностью до шага. В строке  $x = \pi$  значения при  $\alpha = 0,5$  и 0,75 отрицательные, так как фаза уже ушла за  $\cos x$  к  $-\sin x$ .

Этот плавный переход не останавливается на  $\alpha = 1$ . С дальнейшим ростом порядка фаза уезжает ещё сильнее, и к  $\alpha = 2$  кривая садится на  $-\sin x$  (таблица 11). В точке  $x = \pi/2$  значение монотонно опускается от 1 при  $\alpha = 0$  до  $-1$  при  $\alpha = 2$ , проходя через ноль около  $\alpha = 1$ .

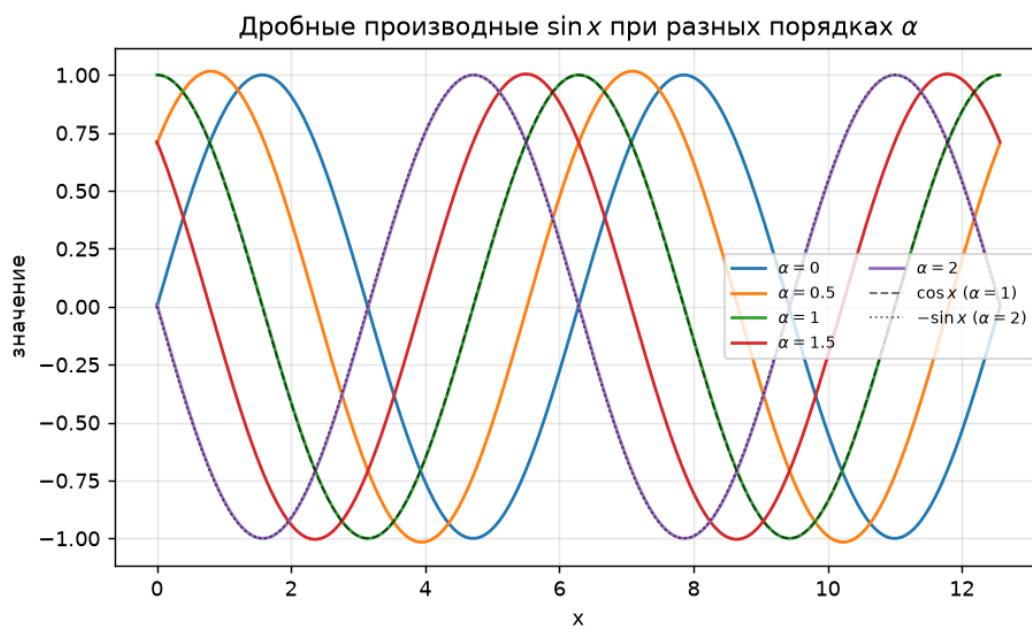


Рисунок 2 – Дробные производные  $\sin x$  при  $\alpha = 0, 0,5, 1, 1,5, 2$  ( $h = 0,01$ ,  $N = 500$ ); пунктиром показаны эталоны  $\cos x$  и  $-\sin x$

Таблица 10 – Значения дробной производной  $\sin x$  при  $h = 0,01$ ,  $N = 500$

| $x$   | $f(x)$ | $D^{0,25}$ | $D^{0,5}$ | $D^{0,75}$ | $D^1$  |
|-------|--------|------------|-----------|------------|--------|
| 0,000 | 0,000  | 0,380      | 0,705     | 0,923      | 1,000  |
| 0,785 | 0,707  | 0,940      | 1,016     | 0,933      | 0,711  |
| 1,571 | 1,000  | 0,950      | 0,732     | 0,397      | 0,005  |
| 2,356 | 0,707  | 0,403      | 0,020     | -0,371     | -0,704 |
| 3,142 | 0,000  | -0,380     | -0,705    | -0,923     | -1,000 |
| 4,712 | -1,000 | -0,950     | -0,732    | -0,397     | -0,005 |
| 6,283 | 0,000  | 0,380      | 0,705     | 0,923      | 1,000  |

Таблица 11 – Значения дробной производной  $\sin x$  для верхних порядков,  $h = 0,01$ ,  $N = 500$

| $x$   | $f(x)$ | $D^{1,25}$ | $D^{1,5}$ | $D^{1,75}$ | $D^2$  |
|-------|--------|------------|-----------|------------|--------|
| 0,000 | 0,000  | 0,926      | 0,712     | 0,390      | 0,010  |
| 0,785 | 0,707  | 0,384      | 0,002     | -0,378     | -0,700 |
| 1,571 | 1,000  | -0,383     | -0,708    | -0,924     | -1,000 |
| 2,356 | 0,707  | -0,925     | -1,004    | -0,929     | -0,714 |
| 3,142 | 0,000  | -0,926     | -0,712    | -0,390     | -0,010 |
| 4,712 | -1,000 | 0,383      | 0,708     | 0,924      | 1,000  |
| 6,283 | 0,000  | 0,926      | 0,712     | 0,390      | 0,010  |

## 4.4 Проверка на целых порядках

Целые порядки удобны тем, что ответ известен заранее: при  $\alpha = 1$  и  $\alpha = 2$  численный результат можно положить рядом с обычной производной и сразу увидеть расхождение. Для четырёх функций такое сравнение  $D^1$  с точной первой производной, вместе с абсолютной и относительной ошибками, собрано в таблице 12.

Таблица 12 – Сравнение  $D^1$  с классической первой производной,  $h = 0,01$ ,  $N = 500$

| Функция  | $x$ | ГЛ      | Точная  | Абс. ошибка | Отн. ошибка |
|----------|-----|---------|---------|-------------|-------------|
| $x^2$    | 2,3 | 4,590   | 4,600   | 0,010       | 0,0022      |
| $x^2$    | 5,0 | 9,990   | 10,000  | 0,010       | 0,0010      |
| $e^x$    | 0,5 | 1,641   | 1,649   | 0,008       | 0,0050      |
| $e^x$    | 5,0 | 147,674 | 148,413 | 0,740       | 0,0050      |
| $\sin x$ | 1,4 | 0,175   | 0,170   | 0,005       | 0,0049      |
| $\cos x$ | 3,2 | 0,053   | 0,058   | 0,005       | 0,0050      |

Для  $x^2$  абсолютная ошибка ровно 0,010 в каждой точке. Левая разность даёт  $\frac{x^2 - (x-h)^2}{h} = 2x - h$ , поэтому появляется систематический сдвиг на  $h = 0,01$ . Для  $\sin x$  и  $\cos x$  ошибка около 0,005, порядка  $h/2$ . Для  $e^x$  абсолютная ошибка растёт вместе с функцией, от 0,008 при  $x = 0,5$  до 0,74 при  $x = 5$ , но относительная держится около 0,005. Для сравнения разных функций это и важно, ведь одна абсолютная ошибка мало о чём говорит, и ориентируются на относительную. У самого столбца относительной ошибки есть оговорка: для  $\sin x$  и  $\cos x$  здесь  $|g| < 1$ , знаменатель  $\max(1, |g|)$  обращается в единицу, и приведённое число совпадает с абсолютной ошибкой; как настоящая относительная оно читается только для  $e^x$  при больших  $x$ , где  $|g| \geq 1$ . На рисунке 3 тот же тест показан графически.

Для  $\alpha = 2$  на  $x^2$  результат равен ровно 2,000 во всех точках независимо от  $h$ : вторая разность квадратичной функции точна. Независимость от шага делает этот случай надёжной эталонной проверкой.



Рисунок 3 –  $D^1 \sin x$  по Грюнвальду–Летникову приближает  $\cos x$  с ошибкой порядка  $h$  ( $h = 0,01$ ,  $N = 500$ )

## 4.5 Влияние шага $h$

Чем крупнее шаг  $h$ , тем дальше  $D^1 \sin x$  отходит от эталона  $\cos x$ . Эту зависимость собирает таблица 13 ( $N = 500$ ).

Таблица 13 – Влияние шага  $h$  на ошибку  $D^1 \sin x$  ( $N = 500$ )

| $h$   | $L = Nh$ | Макс. ошибка | Сред. ошибка |
|-------|----------|--------------|--------------|
| 0,100 | 50,0     | 0,0500       | 0,0335       |
| 0,050 | 25,0     | 0,0250       | 0,0167       |
| 0,020 | 10,0     | 0,0100       | 0,0067       |
| 0,010 | 5,0      | 0,0050       | 0,0033       |
| 0,005 | 2,5      | 0,0025       | 0,0017       |
| 0,001 | 0,5      | 0,0005       | 0,0003       |

Зависимость получается почти линейной. При уменьшении  $h$  вдвое ошибка падает вдвое, максимальная ошибка примерно равна  $h/2$ . При  $h = 0,1$  максимальная ошибка 0,05, при  $h = 0,01$  уже 0,005, в десять раз меньше. На рисунке 4 при  $h = 0,5$  кривая заметно сдвинута и ниже по амплитуде, при  $h = 0,02$  почти сливается с эталоном.

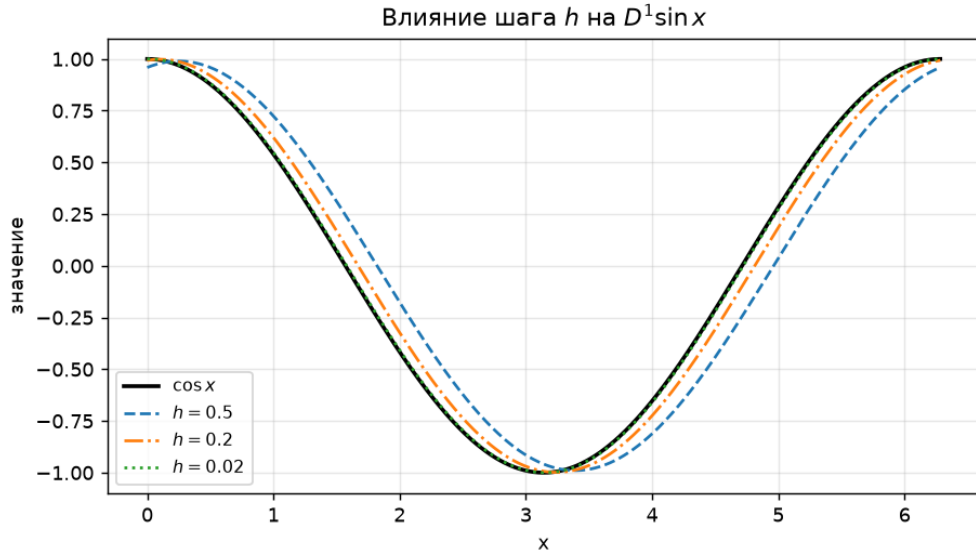


Рисунок 4 – Влияние шага  $h$  на  $D^1 \sin x$  при разной плотности сетки ( $N = 500$ )

## 4.6 Влияние числа членов $N$

Насколько длинной должна быть история? Для целого порядка вопрос не стоит, там работают всего два коэффициента; для дробного же число членов  $N$  определяет точность напрямую. В таблице 14 оно растёт от 5 до 1000 при  $\sin x$ ,  $\alpha = 0,5$  и  $h = 0,01$ , а эталоном служит  $\sin(x + \pi/4)$ .

Таблица 14 – Влияние числа членов  $N$  на ошибку  $D^{0,5} \sin x$  ( $h = 0,01$ )

| $N$  | $L = Nh$ | Макс. ошибка | Сред. ошибка |
|------|----------|--------------|--------------|
| 5    | 0,05     | 1,850        | 1,175        |
| 20   | 0,20     | 0,719        | 0,458        |
| 50   | 0,50     | 0,336        | 0,214        |
| 100  | 1,00     | 0,170        | 0,109        |
| 300  | 3,00     | 0,044        | 0,028        |
| 500  | 5,00     | 0,025        | 0,016        |
| 1000 | 10,0     | 0,008        | 0,005        |

При  $N = 5$  (история  $L = 0,05$ ) ошибка достигает 1,85, потому что сумма обрывается раньше, чем накапливается память функции. С ростом  $N$  ошибка падает медленно.  $N = 100$  даёт 0,17,  $N = 500$  даёт 0,025,  $N = 1000$  даёт 0,008. Причина в медленном затухании коэффициентов  $|c_k| \sim k^{-1,5}$ . Падение не бесконечно: при фиксированном  $h$  рост  $N$  убирает



только ошибку усечения, а под ней остаётся ошибка дискретизации порядка  $h$  (раздел 5.1), и именно в эту полку упёрлась бы таблица при дальнейшем росте  $N$ . На рисунке 5 при  $N = 5$  кривая не похожа на эталон, при  $N = 20$  уже близка, при  $N = 200$  почти ложится на  $\sin(x + \pi/4)$ .

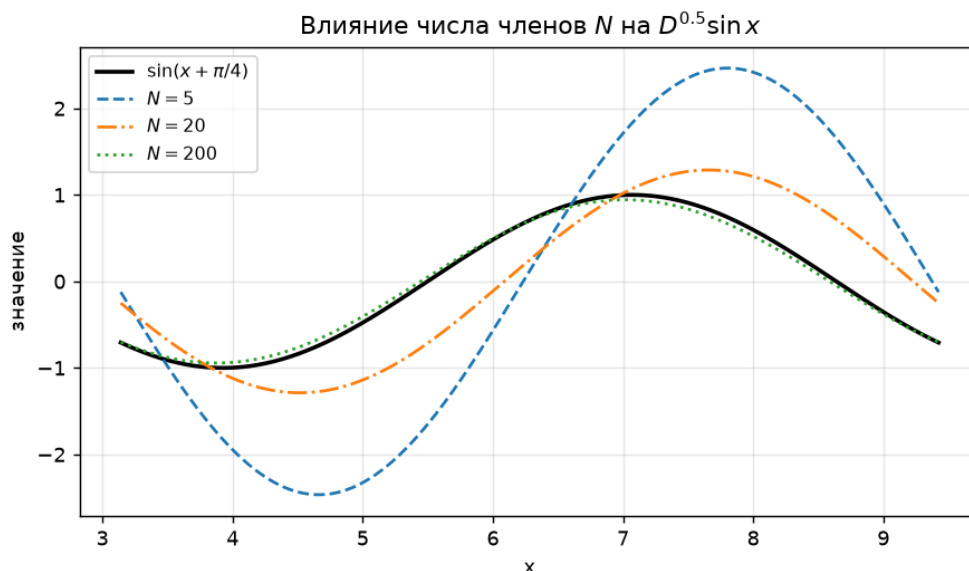


Рисунок 5 – Влияние числа членов  $N$  на  $D^{0,5} \sin x$  ( $h = 0,01$ )

Уменьшать ошибку для дробного порядка только шагом  $h$  недостаточно: при короткой истории  $Nh$  мелкий шаг не помогает.

## 4.7 Память метода

Затухание коэффициентов объясняет разное поведение целого и дробного порядка (рисунок 6). На лог-лог графике коэффициенты при  $\alpha = 0,5$  ложатся на прямую с наклоном  $-1,5 = -(\alpha + 1)$ . При  $\alpha = 1,5$  наклон круче, память короче. При целом  $\alpha = 1$  ненулевой коэффициент всего один, и памяти нет.

## 4.8 Сходимость и порядок точности

Для оценки порядка точности измерялось падение ошибки при уменьшении  $h$  в фиксированной точке. Для  $\sin x$ ,  $\alpha = 1$ ,  $x = 1$  эталон  $\cos 1 \approx 0,5403$  (таблица 15).

При уменьшении шага вдвое ошибка падает примерно вдвое (отношение  $\approx 2$ ), то есть метод имеет первый порядок точности,  $O(h)$ . Это согла-

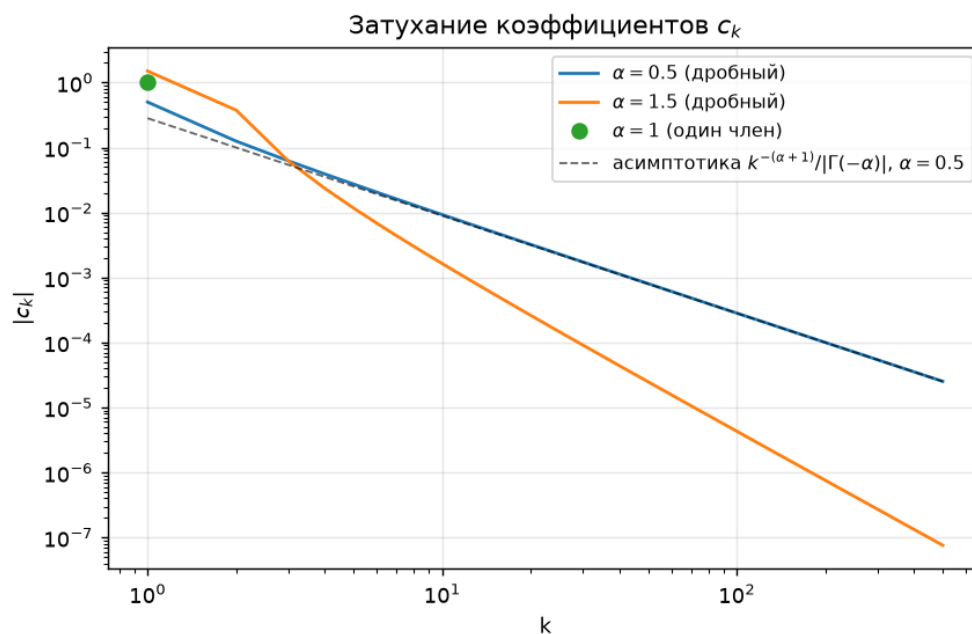


Рисунок 6 – Затухание коэффициентов  $|c_k|$  для дробного и целого порядка

Таблица 15 – Сходимость по  $h$  для  $D^1 \sin x$  в точке  $x = 1$

| $h$     | Ошибка | Отношение |
|---------|--------|-----------|
| 0,200   | 0,0803 | —         |
| 0,100   | 0,0411 | 1,95      |
| 0,050   | 0,0208 | 1,98      |
| 0,025   | 0,0105 | 1,99      |
| 0,0125  | 0,0052 | 1,99      |
| 0,00625 | 0,0026 | 2,00      |

соединяется с известными результатами о порядке аппроксимации дискретного дробного исчисления [6, 7, 3]. На рисунке 7 обе зависимости (для  $\sin x$  при  $\alpha = 1$  и для  $x^2$  при  $\alpha = 0,5$  в режиме  $a = 0$ ) ложатся в прямые с наклоном 1. Дробный случай ведёт себя так же, как целый, что подтверждает корректность реализации для дробного порядка.

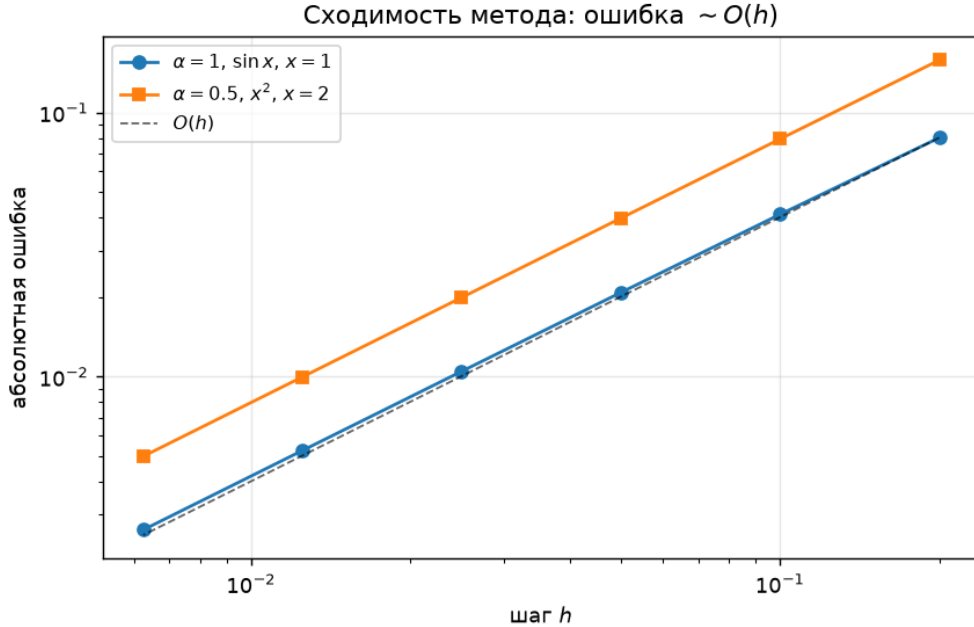


Рисунок 7 – Сходимость по шагу  $h$  для целого и дробного порядка

## 4.9 Степенные функции и логарифм

Для  $x^2$  дробная производная в режиме  $a = 0$  показана на рисунке 8. При  $\alpha = 0,5$  численная кривая ложится на формулу через гамма-функцию, при  $\alpha = 1$  приближает  $2x$  с систематическим сдвигом левой разности.

Для  $x^3$  закономерность степенных функций сохраняется (рисунок 9, таблица 16). Расчёт ведётся в режиме  $a = 0$ . При  $\alpha = 2$  получаются значения вида  $6x - 0,06$ . Точная вторая производная равна  $6x$ , а сдвиг  $0,06 = 6h$  появляется потому, что для кубической функции вторая разность уже не точна. При  $\alpha = 1$  результат равен  $3x^2 - 3xh + h^2$ . Дробный порядок  $D^{0,5}$  согласуется с формулой (11). При  $x = 1$  значение  $1,794$  близко к точному  $\Gamma(4)/\Gamma(3,5) \approx 1,805$ .

Логарифм даёт пример функции с ограниченной областью (рисунок 10). На  $[6, 10]$  при  $h = 0,01$ ,  $N = 500$  условие  $x - Nh > 0$  выполнено,

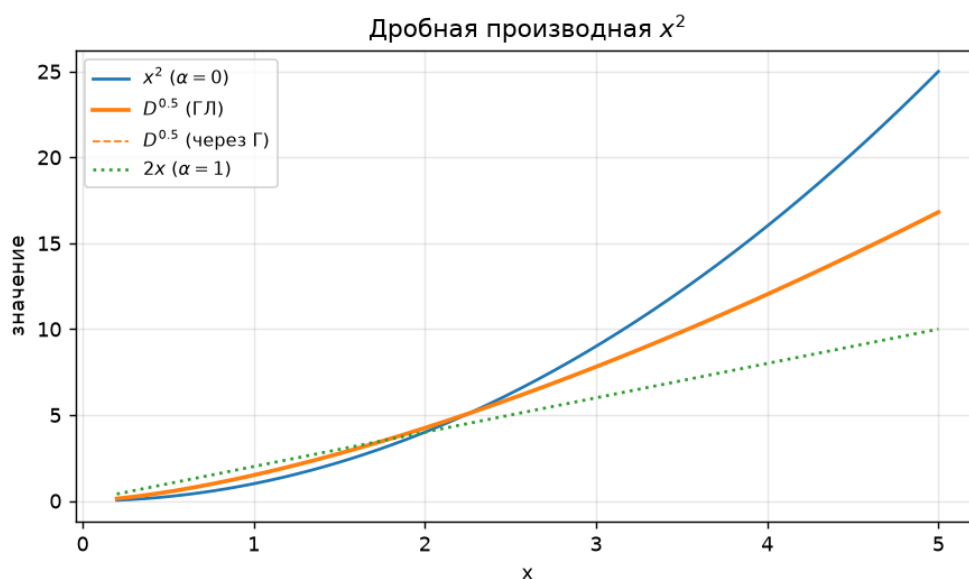


Рисунок 8 – Дробная производная  $x^2$  в режиме  $a = 0$  при нескольких порядках  $\alpha$  ( $h = 0,01$ )

Таблица 16 – Значения дробной производной  $x^3$  в режиме  $a = 0$  при  $N = \lfloor x/h \rfloor$  и  $h = 0,01$

| $x$ | $f(x)$ | $D^{0,5}$ | $D^1$  | $D^{1,5}$ | $D^2$ |
|-----|--------|-----------|--------|-----------|-------|
| 1,0 | 1,000  | 1,794     | 2,970  | 4,463     | 5,94  |
| 2,0 | 8,000  | 10,181    | 11,940 | 12,694    | 11,94 |
| 3,0 | 27,000 | 28,085    | 26,910 | 23,365    | 17,94 |
| 4,0 | 64,000 | 57,683    | 47,880 | 36,007    | 23,94 |

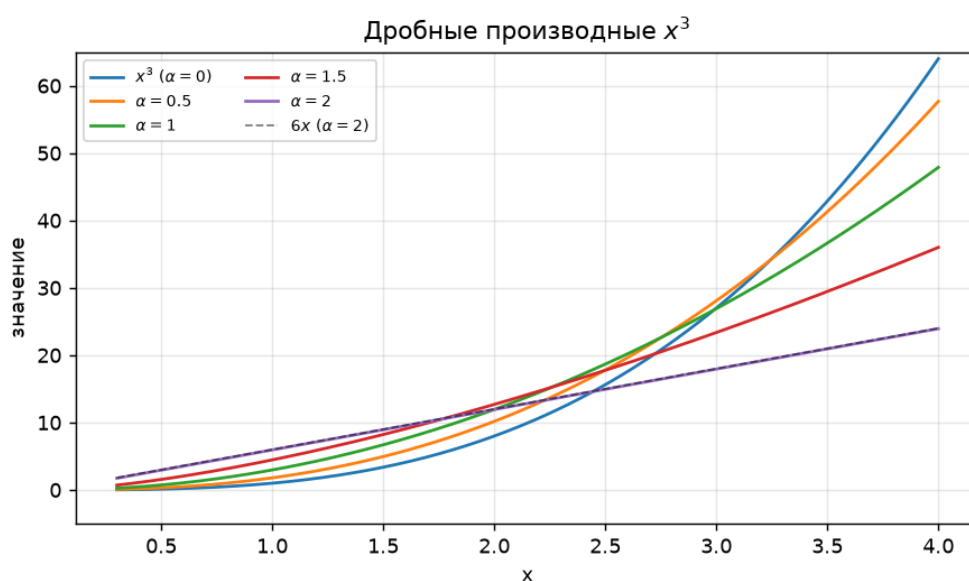


Рисунок 9 – Дробные производные  $x^3$  при росте порядка  $\alpha$  (режим  $a = 0$ ,  $h = 0,01$ )

и  $D^1 \ln x$  ложится на  $1/x$ . Ближе к нулю часть точек  $x - kh$  ушла бы в неположительную область, из-за чего отрезок и держат правее.

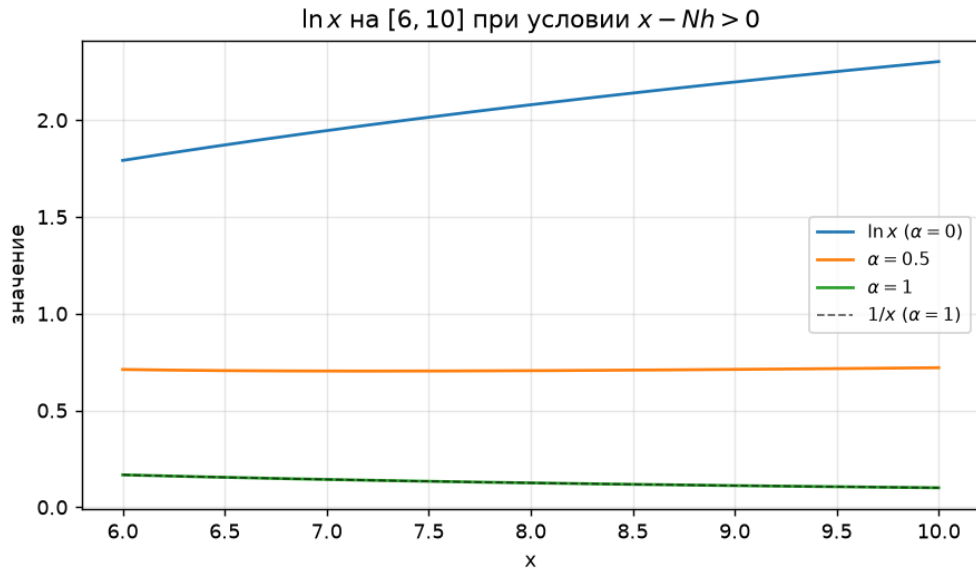


Рисунок 10 – Дробная производная  $\ln x$  на отрезке  $[6, 10]$ , где выполнено условие  $x - Nh > 0$  ( $h = 0,01$ ,  $N = 500$ )

#### 4.10 Экспонента, левая и правая схемы, касательная

Для  $e^x$  форма при разных  $\alpha$  почти не меняется (рисунок 11): все кривые близки к экспоненте, так как  $D^\alpha e^x = e^x$  в форме Лиувилля. Интервал здесь взят умеренный,  $[0, 3]$ , и причина чисто практическая: при больших  $x$  значения растут так быстро, что абсолютная ошибка перестаёт быть маленькой.

Выбор левой схемы заложен в постановку задачи. На рисунке 12 для  $\sin x$  при  $\alpha = 0,5$  левая производная ложится на  $\sin(x + \pi/4)$ , а правая на  $\sin(x - \pi/4)$ . Сдвиг фазы получается в противоположные стороны. Для дробного порядка левая и правая производные дают два разных числа.

У  $\tan x$  есть разрыв в  $\pi/2$ , и около него значения растут неограниченно (рисунок 13). Уже на отрезке от 0,2 до 1,3 и функция, и её дробная производная резко уходят вверх к правому краю. Метод формально работает, но интервал держат в стороне от особой точки. Устойчивость, как видно, упирается не только в параметры схемы, но и в саму функцию.

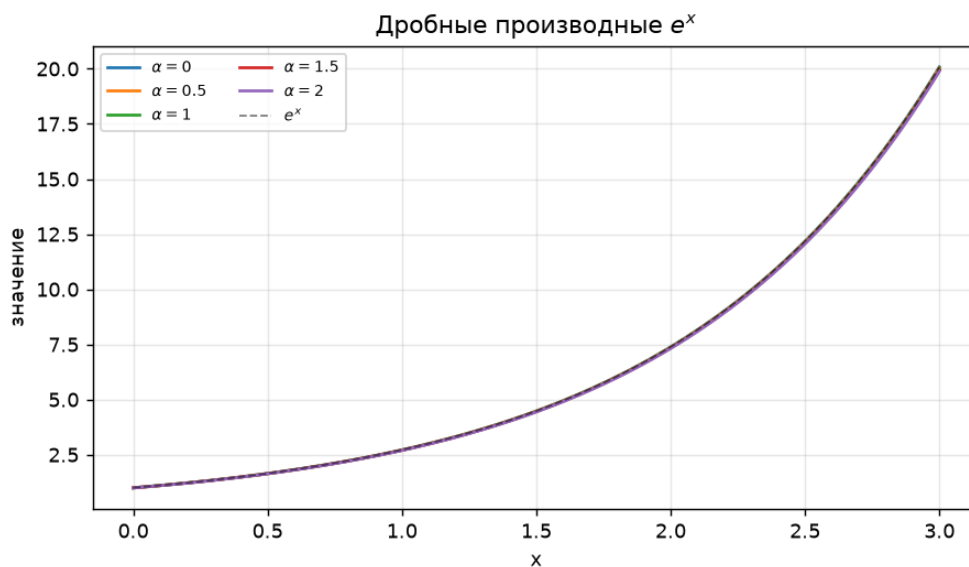


Рисунок 11 – Дробные производные  $e^x$  почти совпадают с самой функцией ( $h = 0,01$ ,  $N = 500$ )

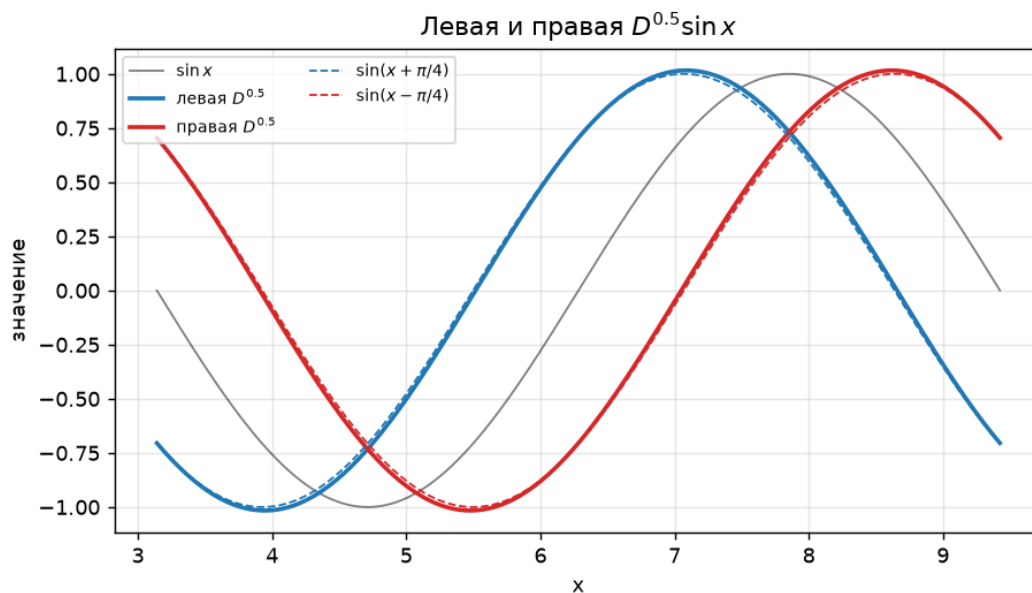


Рисунок 12 – Левая и правая  $D^{0.5} \sin x$  со сдвигом фазы в разные стороны,  $\sin(x \pm \pi/4)$  ( $h = 0,01$ ,  $N = 500$ )

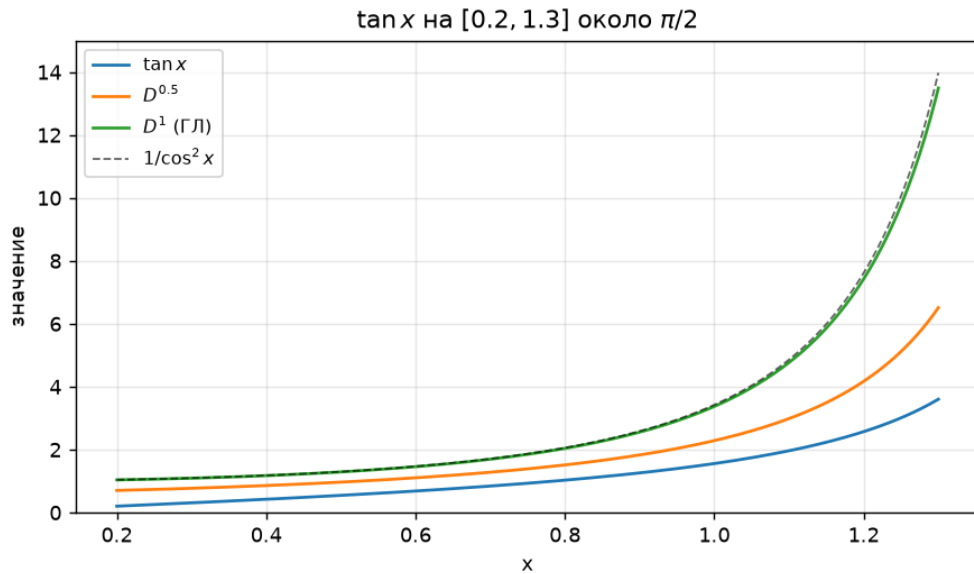


Рисунок 13 – Поведение  $\tan x$  и его дробной производной вблизи разрыва в точке  $\pi/2$  ( $h = 0,01$ ,  $N = 100$ , память  $L = 1$ )

## 4.11 Карта дробных производных

Если построить  $D^\alpha \sin x$  для непрерывной сетки порядков, получится сводная карта значений (рисунок 14). По горизонтали отложен  $x$ , по вертикали порядок  $\alpha$  от 0 до 2, цветом показано значение производной. Максимумы и минимумы идут наклонными полосами. При росте  $\alpha$  картина равномерно сдвигается влево, и сдвиг линейен по  $\alpha$ .

# 5 Точность, устойчивость и пути её повышения

В этой главе метод рассматривается со стороны самих чисел: какие свойства коэффициентов проверяемы, где у точности есть предел и как этот предел частично обойти.

## 5.1 Аналитическая оценка порядка аппроксимации

Первый порядок  $O(h)$  из таблиц сходимости (раздел 4.8) можно получить и аналитически, не только численно. Производящая функция ко-

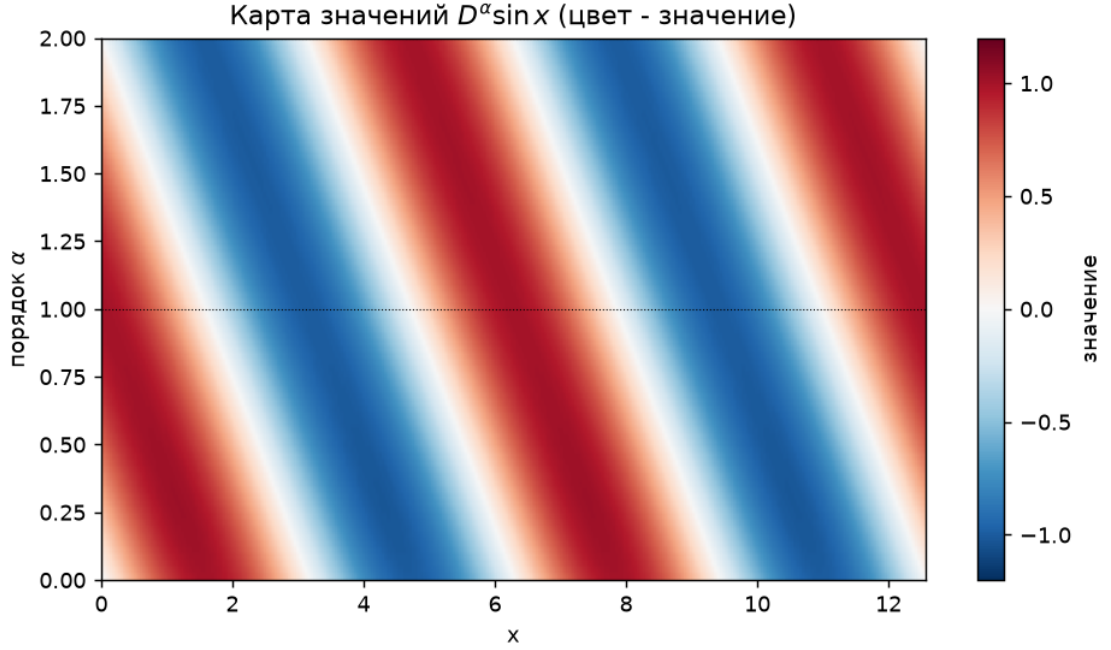


Рисунок 14 – Карта значений  $D^\alpha \sin x$  на плоскости  $(x, \alpha)$ , цвет соответствует значению производной ( $h = 0,01$ ,  $N = 500$ )

эффицентов равна

$$\sum_{k=0}^{\infty} c_k^{(\alpha)} z^k = \sum_{k=0}^{\infty} (-1)^k \binom{\alpha}{k} z^k = (1 - z)^\alpha. \quad (17)$$

Применим разностный оператор  $\delta_h^\alpha f(x) = \sum_{k \geq 0} c_k^{(\alpha)} f(x - kh)$  к гармонике  $f(x) = e^{i\omega x}$ . Поскольку  $f(x - kh) = e^{i\omega x} e^{-i\omega kh}$ , сумма сворачивается в производящую функцию при  $z = e^{-i\omega h}$ :

$$\delta_h^\alpha e^{i\omega x} = (1 - e^{-i\omega h})^\alpha e^{i\omega x}, \quad D_h^\alpha e^{i\omega x} = \left( \frac{1 - e^{-i\omega h}}{h} \right)^\alpha e^{i\omega x}. \quad (18)$$

Из разложения  $1 - e^{-i\omega h} = i\omega h \left( 1 - \frac{i\omega h}{2} + O(h^2) \right)$  следует

$$\left( \frac{1 - e^{-i\omega h}}{h} \right)^\alpha = (i\omega)^\alpha \left( 1 - \frac{\alpha}{2} i\omega h + O(h^2) \right) = (i\omega)^\alpha - \frac{\alpha}{2} (i\omega)^{\alpha+1} h + O(h^2). \quad (19)$$

Множитель  $(i\omega)^\alpha$  задаёт символ точной производной  $D^\alpha$ , а  $(i\omega)^{\alpha+1}$  отвечает  $D^{\alpha+1}$  (при нецелом  $\alpha$  берётся главная ветвь степенной функции,  $i^\alpha = e^{i\pi\alpha/2}$ , что отвечает левому оператору в форме Лиувилля и согласуется с фазовым



сдвигом (12)). Поэтому для гладкой функции

$$D_h^\alpha f(x) = D^\alpha f(x) - \frac{\alpha}{2} h D^{\alpha+1} f(x) + O(h^2). \quad (20)$$

Отсюда сразу виден первый порядок  $O(h)$ , причём ведущая ошибка пропорциональна  $D^{\alpha+1} f$ . При  $\alpha = 1$  это  $-\frac{h}{2} f''$ , и для  $\sin x$  в точке  $x = 1$  по модулю получается  $\frac{h}{2} |\sin 1| \approx 0,42 h$ . Таблица 15 это и показывает: при  $h = 0,1$  ошибка  $0,041 \approx 0,42 \cdot 0,1$ . Разложение (20) объясняет и экстраполяцию Ричардсона: она построена так, чтобы погасить именно линейный по  $h$  член.

У этой оценки есть важная оговорка. Разложение (20) получено для полной суммы с бесконечной историей, то есть в нём учтена только ошибка дискретизации. При конечном  $N$  к ней добавляется ошибка усечения, которую символьный анализ не видит: дальние коэффициенты  $|c_k| \sim k^{-(\alpha+1)}$  отбрасываются, и их вклад убывает как  $N^{-\alpha}$  (раздел 4.6). Полная погрешность складывается из двух независимых частей, дискретизации  $O(h)$  и усечения  $O(N^{-\alpha})$ . В режиме нижнего предела  $a = 0$  степенная история обрывается естественно при  $N = \lfloor x/h \rfloor$ , член усечения исчезает, и остаётся чистый  $O(h)$ ; в режиме конечной памяти  $L = Nh$  заметны оба слагаемых, и уменьшать одно только  $h$  при коротком  $N$  бесполезно.

## 5.2 Проверка через сумму коэффициентов

У коэффициентов есть свойство, не зависящее ни от функции, ни от точки. Сумма всех коэффициентов равна

$$\sum_{k=0}^{\infty} c_k^{(\alpha)} = \sum_{k=0}^{\infty} (-1)^k \binom{\alpha}{k} = (1-1)^\alpha = 0 \quad (\alpha > 0). \quad (21)$$

Тогда частичные суммы  $S_n = \sum_{k=0}^n c_k$  обязаны стремиться к нулю (таблица 17). Они убывают к нулю медленно: для  $\alpha = 0,5$  при  $n = 10$  сумма ещё  $0,176$ , при  $n = 100$  уже  $0,056$ , а при  $n = 500$  всего  $0,025$ . Это согласуется с медленным затуханием коэффициентов и служит независимым тестом, ведь при ошибке в рекуррентной формуле суммы не стремились бы к нулю.

Таблица 17 – Частичные суммы коэффициентов  $S_n = \sum_{k=0}^n c_k$  (стремятся к нулю при  $\alpha > 0$ )

| $n$ | $\alpha = 0,25$ | $\alpha = 0,5$ | $\alpha = 0,75$ |
|-----|-----------------|----------------|-----------------|
| 1   | 0,750           | 0,500          | 0,250           |
| 5   | 0,536           | 0,246          | 0,081           |
| 10  | 0,455           | 0,176          | 0,049           |
| 50  | 0,306           | 0,080          | 0,015           |
| 100 | 0,258           | 0,056          | 0,009           |
| 500 | 0,173           | 0,025          | 0,003           |

### 5.3 Порядок усечения по числу членов

На лог-лог графике (рисунок 15) ошибка усечения против  $N$  ложится почти на прямую, наклон растёт от примерно 0,8 при малых  $N$  до примерно 1,3 при больших. Это быстрее, чем даёт теоретическая оценка короткой памяти  $N^{-\alpha} = N^{-0,5}$  (пунктир на графике): для  $\sin x$  помогает то, что значения ограничены и знакопеременны, и дальние слагаемые частично гасят друг друга.

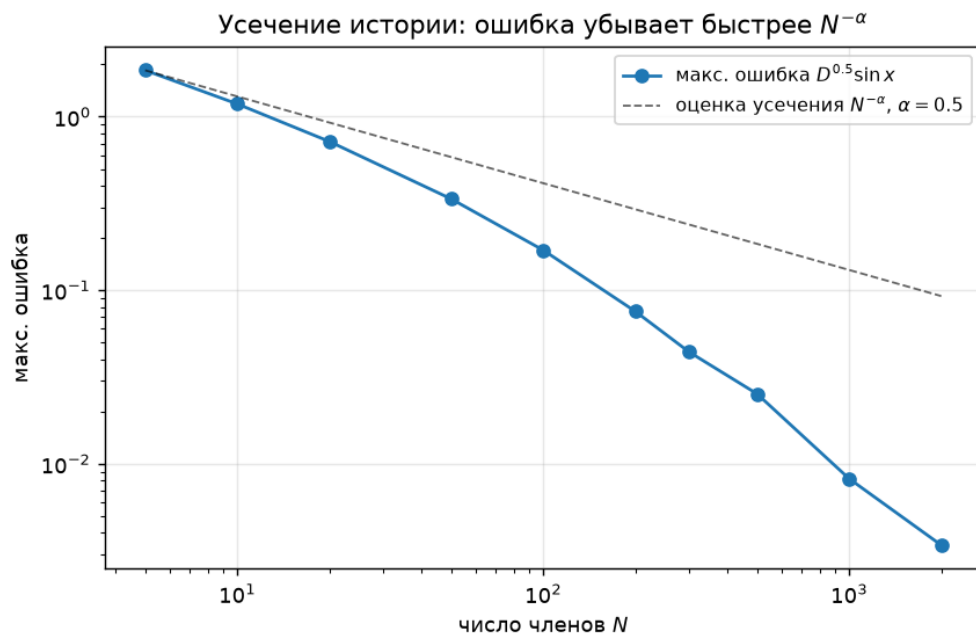


Рисунок 15 – Ошибка усечения  $D^{0,5} \sin x$  в зависимости от числа членов  $N$  в лог-лог осях ( $h = 0,01$ )

## 5.4 Предел точности и порог округления

Уменьшать шаг бесконечно нельзя: деление на  $h^\alpha$  усиливает ошибки округления. Для  $D^1 \sin x$  в точке  $x = 1$  шаг  $h$  менялся от 0,1 до  $10^{-11}$  (таблица 18, рисунок 16).

Таблица 18 – Ошибка  $D^1 \sin x$  в точке  $x = 1$  при уменьшении шага  $h$

| $h$        | Ошибка              |
|------------|---------------------|
| $10^{-1}$  | $4,1 \cdot 10^{-2}$ |
| $10^{-3}$  | $4,2 \cdot 10^{-4}$ |
| $10^{-5}$  | $4,2 \cdot 10^{-6}$ |
| $10^{-7}$  | $4,1 \cdot 10^{-8}$ |
| $10^{-8}$  | $8,1 \cdot 10^{-9}$ |
| $10^{-9}$  | $5,8 \cdot 10^{-8}$ |
| $10^{-11}$ | $1,2 \cdot 10^{-6}$ |

Сначала ошибка падает пропорционально  $h$ . Около  $h \approx 10^{-8}$  она достигает минимума примерно  $8 \cdot 10^{-9}$ , дальше снова растёт. При  $h = 10^{-11}$  ошибка уже  $1,2 \cdot 10^{-6}$ . Слева работает дискретизация ( $\sim h/2$ ), справа округление ( $\sim \varepsilon/h$ , где  $\varepsilon$  — машинная точность), минимум стоит около  $h \sim \sqrt{\varepsilon} \approx 10^{-8}$ . Базовый  $h = 0,01$  находится далеко в безопасной зоне.

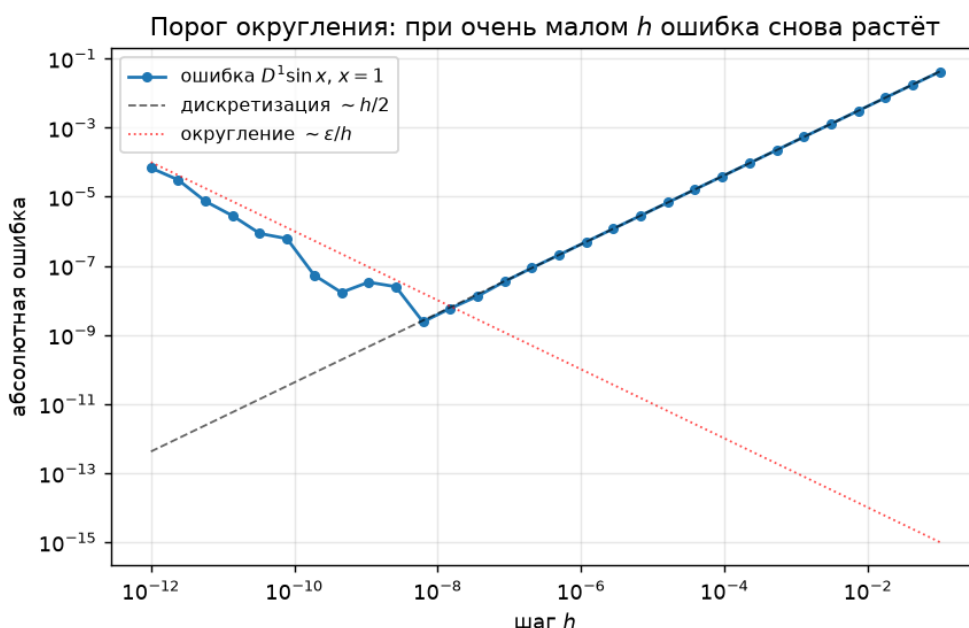


Рисунок 16 – Порог округления ошибки  $D^1 \sin x$  в точке  $x = 1$

## 5.5 Повышение порядка: экстраполяция Ричардсона

Раз ошибка ведёт себя как  $Ch + O(h^2)$ , линейный член можно погасить. Комбинация результатов на шагах  $h$  и  $h/2$

$$R(h) = 2D_{h/2}^\alpha f(x) - D_h^\alpha f(x) \quad (22)$$

убирает слагаемое  $Ch$  и оставляет  $O(h^2)$  [9]. Проверка проведена на  $D^1 \sin x$  в точке  $x = 1$  (таблица 19) и на дробном порядке  $D^{0,5}x^2$  в точке  $x = 2$  (таблица 20).

Таблица 19 – Обычная схема против экстраполяции Ричардсона для  $D^1 \sin x$ ,  $x = 1$

| $h$     | Ошибка обычн. | Отнош. | Ошибка Ричардсон | Отнош. |
|---------|---------------|--------|------------------|--------|
| 0,200   | 0,0803        | —      | 0,00200          | —      |
| 0,100   | 0,0411        | 1,95   | 0,00048          | 4,21   |
| 0,050   | 0,0208        | 1,98   | 0,00012          | 4,11   |
| 0,025   | 0,0105        | 1,99   | 0,000029         | 4,06   |
| 0,0125  | 0,0052        | 1,99   | 0,0000072        | 4,03   |
| 0,00625 | 0,0026        | 2,00   | 0,0000018        | 4,01   |

Таблица 20 – Экстраполяция Ричардсона для дробного порядка  $D^{0,5}x^2$ ,  $x = 2$  (режим  $a = 0$ )

| $h$   | Ошибка обычн. | Ошибка Ричардсон    | Отнош. Ричардсона |
|-------|---------------|---------------------|-------------------|
| 0,040 | 0,0319        | $3,3 \cdot 10^{-5}$ | —                 |
| 0,020 | 0,0159        | $8,3 \cdot 10^{-6}$ | 4,00              |
| 0,010 | 0,0080        | $2,1 \cdot 10^{-6}$ | 4,00              |
| 0,005 | 0,0040        | $5,2 \cdot 10^{-7}$ | 4,00              |

У обычной схемы при уменьшении  $h$  вдвое ошибка падает в 2 раза (первый порядок), у экстраполяции в 4 раза (второй порядок,  $O(h^2)$ ). При  $\alpha = 1$  уже при  $h = 0,2$  экстраполяция даёт ошибку 0,002 против 0,08 у обычной схемы. Платой за это становится второй расчёт с половинным шагом. Для дробного порядка  $\alpha = 0,5$  отношение тоже близко к 4 (таблица 20). Рисунки 17 и 18 показывают это в лог-лог осях.

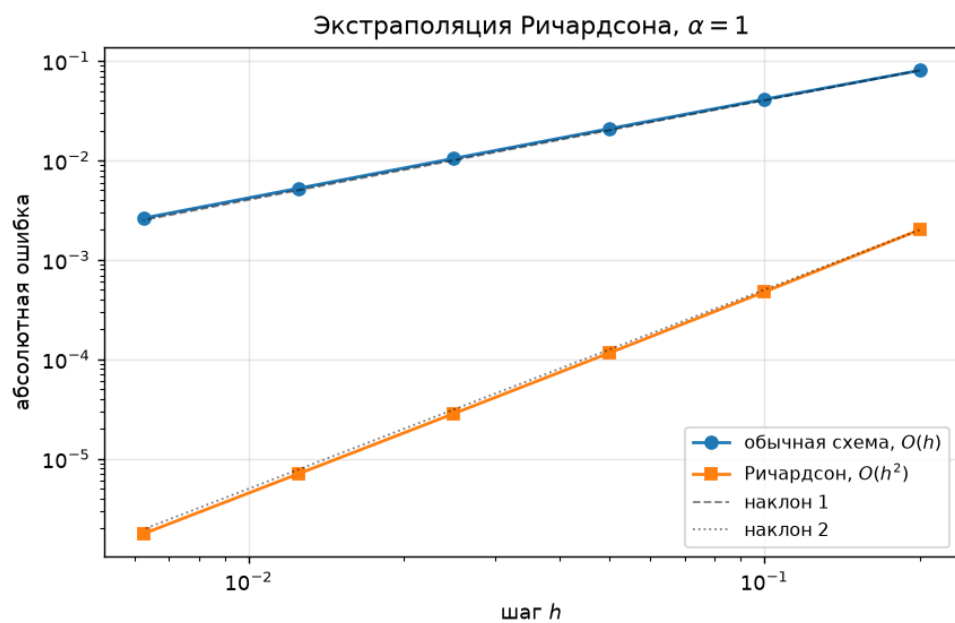


Рисунок 17 – Экстраполяция Ричардсона против обычной схемы для  $D^1 \sin x$

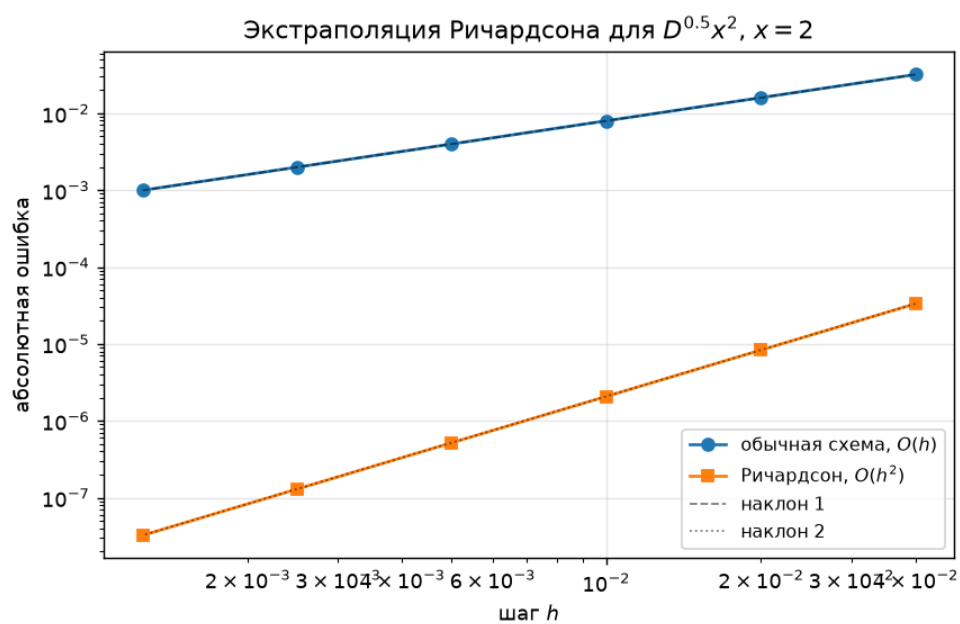


Рисунок 18 – Экстраполяция Ричардсона для  $D^{0.5} x^2$  в режиме  $a = 0$

## 5.6 Нижний предел против конечной памяти

Покажем численно, что режимы из раздела 2.3 дают разные значения. Возьмём  $D^{0,5}x^2$  в точке  $x = 2$ ,  $h = 0,01$  и посчитаем тремя способами (таблица 21). Точное значение в режиме  $a = 0$  равно  $\Gamma(3)/\Gamma(2,5) \cdot 2^{1,5} \approx 4,255$ .

Таблица 21 – Один и тот же  $D^{0,5}x^2$  в точке  $x = 2$  при разных режимах,  $h = 0,01$

| Режим                     | $N$ | Значение | Эталон $a = 0$ |
|---------------------------|-----|----------|----------------|
| Нижний предел $a = 0$ .   | 200 | 4,247    | 4,255          |
| Конечная память $L = 5$ . | 500 | 3,944    | —              |
| Фиксированное $N = 50$ .  | 50  | 4,706    | —              |

Три режима дают 4,247, 3,944 и 4,706, то есть расходятся на десятки процентов. С гамма-формулой совпадает только режим  $a = 0$  (отклонение 0,008, порядка  $h$ ). Режимы конечной памяти считают дробную производную с обрезанной историей, и гамма-формула для них уже не эталон. Поэтому у каждой таблицы в подписи указан свой режим.

## 5.7 Совместное влияние шага и числа членов

Рисунок 19 показывает совместное влияние  $h$  и  $N$  для целого порядка. Цветом отмечен десятичный логарифм максимальной ошибки  $D^1 \sin x$ . Для  $\alpha = 1$  ошибка определяется почти только шагом  $h$  (горизонтальные полосы), потому что коэффициенты обрываются и  $N$  роли не играет.

Для дробного порядка картина другая (рисунок 20). Полосы наклонные, ошибка  $D^{0,5} \sin x$  падает и при уменьшении  $h$ , и при росте  $N$ . Минимум достигается в правом нижнем углу, где одновременно мелкий шаг и длинная история.

## 5.8 Сдвинутая схема Грюнвальда

В стандартной схеме самый правый узел суммы совпадает с самой точкой  $x$  (сдвиг  $p = 0$ ). Сдвинутая схема Грюнвальда берёт узлы  $x - (k -$

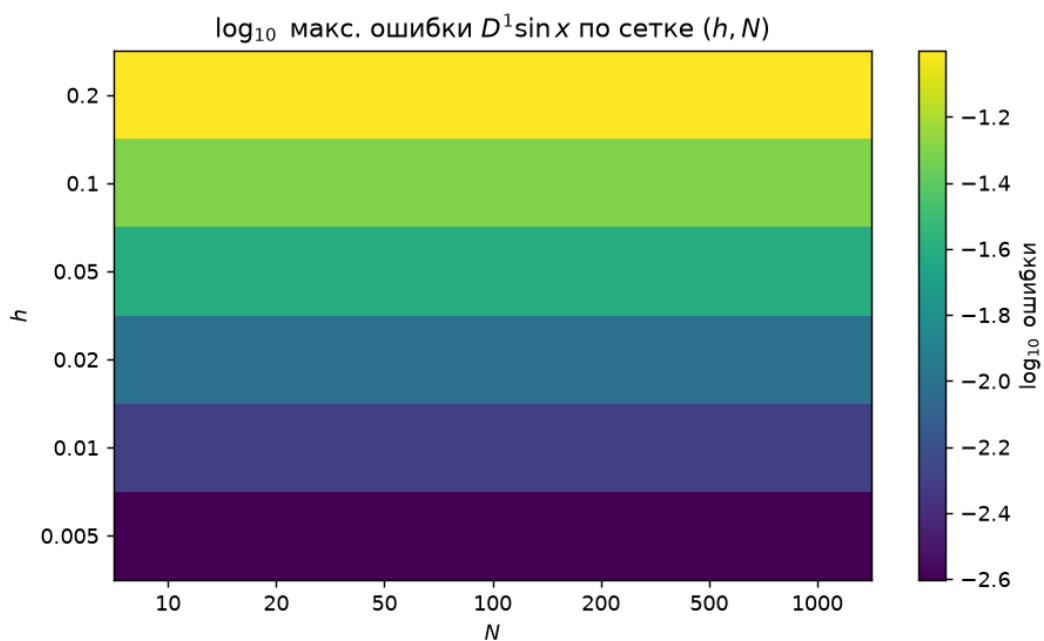


Рисунок 19 – Карта максимальной ошибки  $D^1 \sin x$  по сетке  $(h, N)$

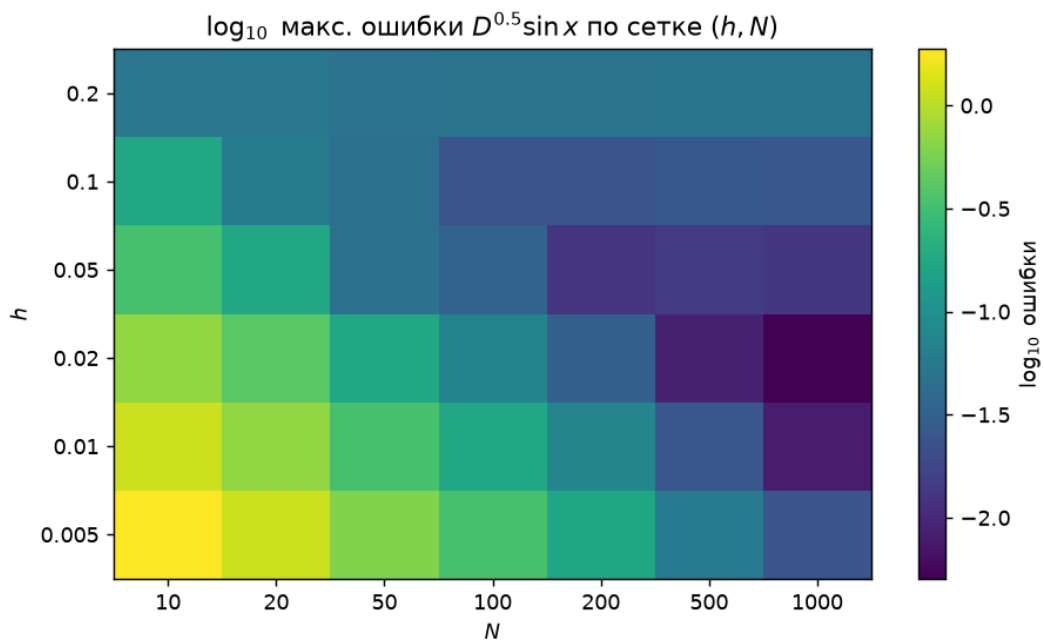


Рисунок 20 – Карта максимальной ошибки  $D^{0.5} \sin x$  по сетке  $(h, N)$

$p)h$ , сдвигая шаблон на  $p$  шагов вправо [7]:

$$D_h^{\alpha,p} f(x) = \frac{1}{h^\alpha} \sum_{k=0}^N c_k^{(\alpha)} f(x - (k - p)h). \quad (23)$$

Коэффициенты те же; меняется только положение узлов, и при целом сдвиге  $p = 1$  верхний узел равен  $x + h$ . Такая схема тоже имеет первый порядок  $O(h)$ , но её ценность в другом. При дискретизации уравнений с дробной производной по пространству именно сдвиг  $p = 1$  делает разностную матрицу устойчивой, тогда как несдвинутая схема ( $p = 0$ ) даёт неустойчивую дискретизацию [7].

На отдельном значении производной сдвиг почти незаметен, зато при дискретизации уравнений становится принципиальным. Аномальная диффузия, упомянутая во введении, описывается уравнением с дробной производной по пространству,  $\partial_t u = K D_x^\beta u$  при  $1 < \beta < 2$ . Если оператор  $D_x^\beta$  заменить обычной суммой Грюнвальда–Летникова ( $p = 0$ ), собственные значения разностной матрицы получают положительную вещественную часть, и явная схема по времени расходится; сдвиг на один узел ( $p = 1$ ) делает эти вещественные части неположительными и возвращает устойчивость [7]. Так малозаметный на одной точке параметр  $p$  определяет устойчивость всей схемы; ради этого сдвинутый вариант и включён в работу рядом со стандартным.

Сравнение на  $D^{0,5}x^2$  в точке  $x = 2$  (режим  $a = 0$ , эталон  $\approx 4,255$ ) приведено в таблице 22. Обе схемы сходятся с первым порядком, отношение ошибок близко к 2. При одном и том же  $h$  сдвинутая схема даёт примерно втрое большую ошибку, и это плата за сдвиг шаблона. В лог-лог осях обе зависимости ложатся в прямые с наклоном 1 (рисунок 21).



Таблица 22 – Стандартная и сдвинутая схемы Грюнвальда для  $D^{0,5}x^2$ ,  $x = 2$  (режим  $a = 0$ )

| $h$    | Ошибка ГЛ ( $p = 0$ ) | Отнош. | Ошибка сдвиг ( $p = 1$ ) | Отнош. |
|--------|-----------------------|--------|--------------------------|--------|
| 0,0400 | 0,0318                | —      | 0,0961                   | —      |
| 0,0200 | 0,0159                | 2,00   | 0,0480                   | 2,00   |
| 0,0100 | 0,0080                | 2,00   | 0,0240                   | 2,00   |
| 0,0050 | 0,0040                | 2,00   | 0,0120                   | 2,00   |
| 0,0025 | 0,0020                | 2,00   | 0,0060                   | 2,00   |

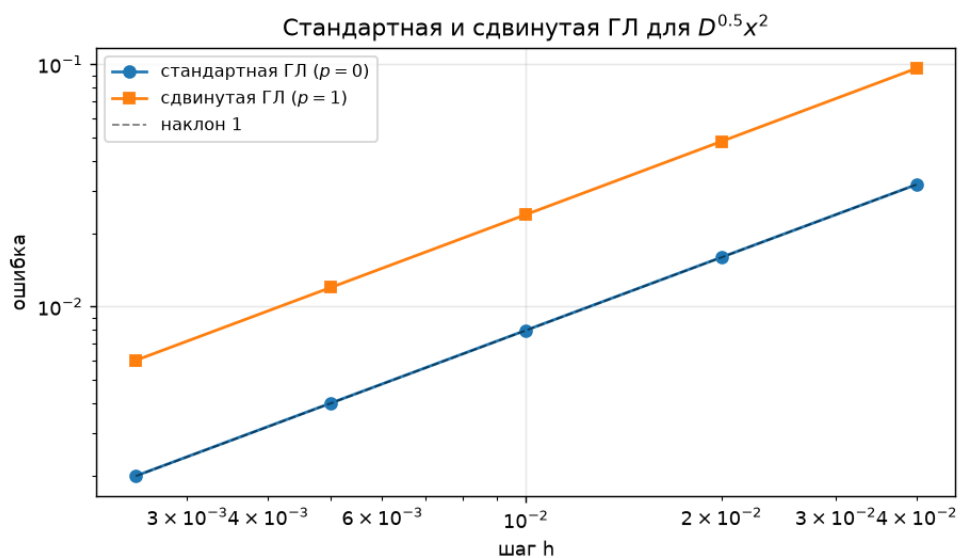


Рисунок 21 – Сходимость стандартной и сдвинутой схем Грюнвальда для  $D^{0,5}x^2$

## 5.9 Сравнение с производной Капуто

Определение Капуто, введённое в главе 1, на практике считают по схеме L1. Для  $0 < \alpha < 1$  и нижнего предела 0

$$D_C^\alpha f(t_n) \approx \frac{1}{\Gamma(2-\alpha)h^\alpha} \sum_{k=0}^{n-1} b_k (f(t_{n-k}) - f(t_{n-k-1})), \quad b_k = (k+1)^{1-\alpha} - k^{1-\alpha}, \quad (24)$$

где  $t_j = jh$ . В отличие от Грюнвальда–Летникова, схема L1 опирается не на сами значения функции, а на её приращения. Поэтому производная постоянной по Капуто равна нулю, тогда как у Грюнвальда–Летникова и Римана–Лиувилля  $D^\alpha 1 = x^{-\alpha}/\Gamma(1-\alpha) \neq 0$ . На гладких функциях схема L1 имеет порядок аппроксимации  $O(h^{2-\alpha})$  [2], то есть при  $0 < \alpha < 1$  точнее первого порядка  $O(h)$  стандартной схемы Грюнвальда–Летникова; в работе она привлекается лишь для иллюстрации граничного слагаемого, отличающего Капуто от Римана–Лиувилля.

Определения связаны соотношением  $D_{RL}^\alpha f = D_C^\alpha f + f(0)x^{-\alpha}/\Gamma(1-\alpha)$ . Численное сравнение на  $e^x$  при  $\alpha = 0,5$ ,  $h = 0,005$  (таблица 23) его подтверждает: разность Грюнвальда–Летникова и Капуто близка к слагаемому  $f(0)x^{-\alpha}/\Gamma(1-\alpha)$  с  $f(0) = 1$ . С ростом  $x$  это слагаемое убывает как  $x^{-\alpha}$ , и определения сближаются (рисунок 22). Для функций с  $f(0) = 0$ , например  $x^2$ , обе схемы совпадают с формулой через гамма-функцию.

Таблица 23 – Грюнвальд–Летников и Капуто для  $D^{0,5}e^x$ ,  $h = 0,005$

| $x$ | ГЛ ( $a = 0$ ) | Капуто (L1) | ГЛ – Капуто | $f(0)x^{-\alpha}/\Gamma(1-\alpha)$ |
|-----|----------------|-------------|-------------|------------------------------------|
| 0,5 | 1,920          | 1,125       | 0,795       | 0,798                              |
| 0,9 | 2,609          | 2,017       | 0,591       | 0,595                              |
| 1,3 | 3,767          | 3,277       | 0,490       | 0,495                              |
| 1,7 | 5,543          | 5,117       | 0,426       | 0,433                              |
| 2,1 | 8,215          | 7,835       | 0,380       | 0,389                              |
| 2,5 | 12,215         | 11,873      | 0,342       | 0,357                              |

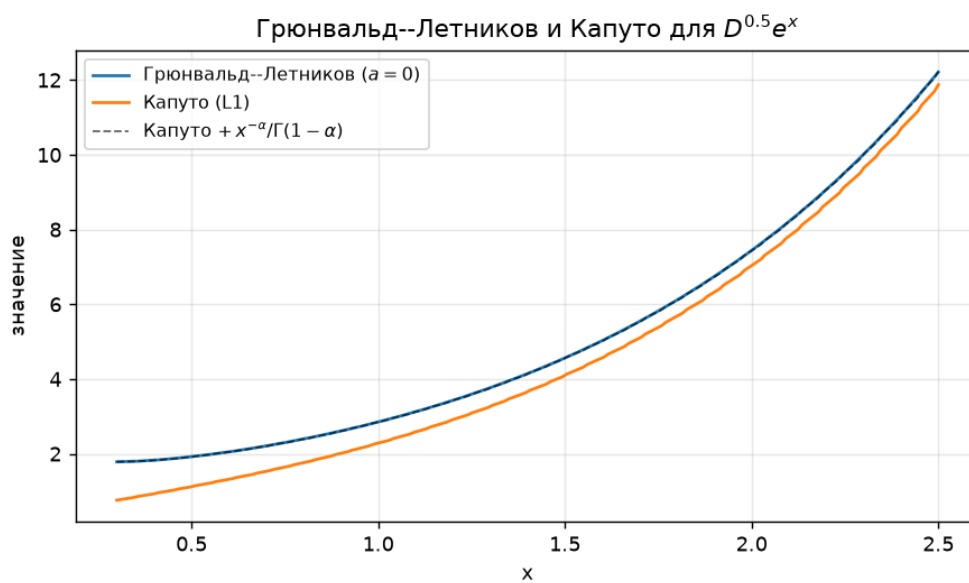


Рисунок 22 – Грюнвальд–Летников и Капуто для  $D^{0,5}e^x$  ( $h = 0,005$ ); пунктиром показана сумма Капуто и граничной поправки

## Заключение

Главным результатом работы стала сводная таблица дробных производных элементарных функций (таблица 7), построенная численным методом Грюнвальда–Летникова и сверенная с замкнутыми формами из справочной таблицы 2. Чтобы её получить, реализовано вычисление дробных производных по формуле Грюнвальда–Летникова и собран небольшой программный комплекс. В него входят модуль расчёта, набор функций, генерация таблиц и графиков, командный запуск, автоматические тесты и веб-интерфейс на React с интерактивным графиком Plotly. Метод хорошо ложится на табличные вычисления, поскольку записывается рекуррентной формулой для коэффициентов и одним матричным умножением для суммы по сетке.

Результаты подтверждены численно. На целых порядках реализация прошла проверку.  $D^1 \sin x$  приближает  $\cos x$  с ошибкой порядка  $h$ ,  $D^2 x^2$  даёт ровно 2, а для  $x^2$  при  $\alpha = 1$  ошибка равна ровно  $h$  из-за смещения левой разности.

Сходимость по шагу имеет первый порядок. При уменьшении  $h$  вдвое ошибка падает вдвое, одинаково для целого ( $\sin x$ ) и дробного ( $x^2$ ,  $\alpha = 0,5$ ) порядка. В режиме нижнего предела  $a = 0$  дробная производная  $x^2$  согласуется с аналитической формулой через гамма-функцию с ошибкой порядка  $h$ . Режимы конечной памяти дают другие значения, и сравнивать их с гамма-формулой уже нельзя.

Параметры  $h$  и  $N$  влияют по-разному. Шаг  $h$  задаёт точность сетки. Ошибка  $D^1 \sin x$  менялась от 0,05 при  $h = 0,1$  до 0,0005 при  $h = 0,001$ . Число  $N$  задаёт длину памяти. Для  $D^{0,5} \sin x$  при  $N = 5$  ошибка была 1,85, а при  $N = 1000$  упала до 0,008. Причина заключается в медленном затухании коэффициентов  $|c_k| \sim k^{-(\alpha+1)}$ .

Углублённый разбор точности дал ещё несколько выводов. Проверка через сумму коэффициентов ( $\sum c_k = 0$ ) подтвердила корректность рекуррентной формулы. У ошибки есть порог около  $h \sim 10^{-8}$  (минимум  $\approx 8 \cdot 10^{-9}$ ), ниже которого округление при делении на  $h^\alpha$  её увеличивает. Экстраполяция Ричардсона поднимает порядок точности до  $O(h^2)$  как для целого, так и для дробного порядка. Сравнение левой и правой схем показало, что для дробного порядка это два разных результата со сдвигом

фазы в противоположные стороны.

Первый порядок  $O(h)$  получен не только численно, но и аналитически: разложение символа схемы даёт  $D_h^\alpha f = D^\alpha f - \frac{\alpha}{2}h D^{\alpha+1}f + O(h^2)$ , и это же разложение объясняет работу экстраполяции Ричардсона. Сдвинутая схема Грюнвальда ( $p = 1$ ) сохраняет порядок  $O(h)$ , но даёт устойчивую дискретизацию для уравнений с дробной производной. Численное сравнение с производной Капуто подтвердило граничное соотношение  $D_{RL}^\alpha f = D_C^\alpha f + f(0)x^{-\alpha}/\Gamma(1-\alpha)$ : определения различаются на слагаемое, убывающее как  $x^{-\alpha}$ , а производная постоянной по Капуто равна нулю.

Есть у метода и ограничения, и о них честнее сказать прямо. Он даёт численное приближение всего первого порядка, и результат заметно зависит от шага, длины истории и поведения самой функции. На гладких функциях ( $\sin x$ ,  $\cos x$ ,  $e^x$ ) он ведёт себя устойчиво. Зато для  $e^x$  абсолютная ошибка растёт вместе со значением функции, для  $\ln x$  приходится следить за условием  $x - Nh > 0$ , а для  $\tan x$  интервал вообще держим подальше от разрыва. Уверенность в правильности складывается из нескольких независимых проверок: совпадения на целых порядках, нулевой суммы коэффициентов и сверки с аналитическими формулами. По строгости это инженерная проверка, и доказательства она не заменяет.

Возможные направления развития включают схемы второго порядка (взвешенные сдвинутые формулы), решение дробных дифференциальных уравнений на основе полученной дискретизации, автоматический подбор  $N$  по заданной точности и ускорение расчёта на равномерной сетке через свёртку.

## Список литературы

- [1] Самко, С. Г. Интегралы и производные дробного порядка и некоторые их приложения / С. Г. Самко, А. А. Килбас, О. И. Маричев. — Минск : Наука и техника, 1987. — 688 с.
- [2] Diethelm, K. The Analysis of Fractional Differential Equations / K. Diethelm. — Berlin : Springer, 2010. — 247 p.
- [3] Diethelm, K. Algorithms for the fractional calculus: a selection of numerical methods / K. Diethelm, N. J. Ford, A. D. Freed, Yu. Luchko // Computer Methods in Applied Mechanics and Engineering. — 2005. — Vol. 194, № 6–8. — P. 743–773.
- [4] Harris, C. R. Array programming with NumPy / C. R. Harris, K. J. Millman, S. J. van der Walt [et al.] // Nature. — 2020. — Vol. 585. — P. 357–362.
- [5] Hunter, J. D. Matplotlib: a 2D graphics environment / J. D. Hunter // Computing in Science & Engineering. — 2007. — Vol. 9, № 3. — P. 90–95.
- [6] Lubich, C. Discretized fractional calculus / C. Lubich // SIAM Journal on Mathematical Analysis. — 1986. — Vol. 17, № 3. — P. 704–719.
- [7] Meerschaert, M. M. Finite difference approximations for fractional advection-dispersion flow equations / M. M. Meerschaert, C. Tadjeran // Journal of Computational and Applied Mathematics. — 2004. — Vol. 172, № 1. — P. 65–77.
- [8] Oldham, K. B. The Fractional Calculus: Theory and Applications of Differentiation and Integration to Arbitrary Order / K. B. Oldham, J. Spanier. — New York : Academic Press, 1974. — 234 p.
- [9] Podlubny, I. Fractional Differential Equations / I. Podlubny. — San Diego : Academic Press, 1999. — 340 p.
- [10] NumPy documentation [Электронный ресурс]. — URL: <https://numpy.org/doc/>

- [11] Python documentation [Электронный ресурс]. — URL:  
<https://docs.python.org/3/>
- [12] React documentation [Электронный ресурс]. — URL: <https://react.dev/>
- [13] Vite documentation [Электронный ресурс]. — URL: <https://vite.dev/>
- [14] Plotly.js documentation [Электронный ресурс]. — URL:  
<https://plotly.com/javascript/>
- [15] Bun documentation [Электронный ресурс]. — URL: <https://bun.sh/docs>

# Приложение А Структура проекта и запуск

Назначение модулей описано в таблице 5. Дерево каталога:

```
CourseWork/  
|-- fractional.py  
|-- functions.py  
|-- experiments.py  
|-- tables.py  
|-- plots.py  
|-- main.py  
|-- requirements.txt  
|-- README.md  
|-- web/  
|   |-- package.json  
|   |-- src/  
|   |   |-- domain/  
|   |   |-- hooks/  
|   |   +-- components/  
|-- tests/  
|   |-- conftest.py  
|   |-- test_coefficients.py  
|   |-- test_fractional_orders.py  
|   |-- test_function_domains.py  
|   |-- test_integer_orders.py  
|   |-- test_alt_schemes.py  
|   +-- test_master_table.py  
|-- graphs/  
|-- tables/  
+-- report/
```

Зависимости Python перечислены в `requirements.txt` и ставятся командой `pip install -r requirements.txt`. Веб-интерфейс устанавливается отдельно из каталога `web/`. Команда `bun install` ставит зависимости, `bun run dev` запускает локальный сайт, `bun run build` собирает статическую версию. Подробные команды запуска приведены в разделе 3.5 и в `README.md`. Кратко: сборка всех таблиц и графиков выполняется командой `python main.py --all`, автоматические тесты запускаются командой `pytest`.

Опубликованный сайт:

<https://course-work-2026.vercel.app/>

Репозиторий проекта:

<https://github.com/SaltyFrappuccino/CourseWork-2026>



# Приложение Б Основные листинги программы

## Б.1 fractional.py

```
import numpy as np
from scipy.special import gamma

def gl_coefficients(alpha, n):
    c = np.empty(n + 1)
    c[0] = 1.0
    if n >= 1:
        k = np.arange(1, n + 1)
        c[1:] = np.cumprod((k - 1 - alpha) / k)
    return c

def gl_point(f, x, alpha, h, n, coeffs=None):
    if coeffs is None:
        coeffs = gl_coefficients(alpha, n)
    k = np.arange(n + 1)
    vals = np.asarray(f(x - k * h), dtype=float)
    return float(coeffs @ vals / h ** alpha)

def gl_grid(f, xs, alpha, h, n):
    coeffs = gl_coefficients(alpha, n)
    xs = np.asarray(xs, dtype=float)
    k = np.arange(n + 1)
    nodes = xs[:, None] - k[None, :] * h
    vals = np.asarray(f(nodes), dtype=float)
    return vals @ coeffs / h ** alpha

def gl_grid_right(f, xs, alpha, h, n):
    coeffs = gl_coefficients(alpha, n)
    xs = np.asarray(xs, dtype=float)
    k = np.arange(n + 1)
    nodes = xs[:, None] + k[None, :] * h
    vals = np.asarray(f(nodes), dtype=float)
    return vals @ coeffs / h ** alpha

def gl_point_from_zero(f, x, alpha, h, coeffs=None):
    n = int(np.floor(x / h + 1e-12))
```

```

    k = np.arange(n + 1)
    if coeffs is None:
        coeffs = gl_coefficients(alpha, n)
    vals = np.asarray(f(x - k * h), dtype=float)
    return float(coeffs[:n + 1] @ vals / h ** alpha)

def gl_grid_from_zero(f, xs, alpha, h):
    xs = np.asarray(xs, dtype=float)
    if xs.size == 0:
        return np.empty(0)
    nmax = int(np.floor(xs.max() / h + 1e-12))
    coeffs = gl_coefficients(alpha, nmax)
    return np.array([gl_point_from_zero(f, float(x), alpha, h, coeffs) for x in xs])

def richardson_point(f, x, alpha, h, n):
    coarse = gl_point(f, x, alpha, h, n)
    fine = gl_point(f, x, alpha, h / 2.0, 2 * n)
    return 2.0 * fine - coarse

def richardson_from_zero(f, x, alpha, h):
    coarse = gl_point_from_zero(f, x, alpha, h)
    fine = gl_point_from_zero(f, x, alpha, h / 2.0)
    return 2.0 * fine - coarse

def coefficient_partial_sums(alpha, n):
    return np.cumsum(gl_coefficients(alpha, n))

def gl_shifted_grid(f, xs, alpha, h, n, p=1.0):
    coeffs = gl_coefficients(alpha, n)
    xs = np.asarray(xs, dtype=float)
    k = np.arange(n + 1)
    nodes = xs[:, None] - (k[None, :] - p) * h
    vals = np.asarray(f(nodes), dtype=float)
    return vals @ coeffs / h ** alpha

def gl_shifted_point_from_zero(f, x, alpha, h, p=1.0):
    n = int(np.floor(x / h + p + 1e-12))
    coeffs = gl_coefficients(alpha, n)
    k = np.arange(n + 1)
    nodes = x - (k - p) * h
    vals = np.asarray(f(nodes), dtype=float)
    return float(coeffs @ vals / h ** alpha)

```

```

def caputo_l1_point(f, x, alpha, h):
    n = int(np.floor(x / h + 1e-12))
    t = np.arange(n + 1) * h
    fv = np.asarray(f(t), dtype=float)
    j = np.arange(1, n + 1)
    b = j ** (1.0 - alpha) - (j - 1) ** (1.0 - alpha)
    diffs = fv[1:] - fv[:-1]
    return float(np.dot(b, diffs[::-1]) / (gamma(2.0 - alpha) * h ** alpha))

def caputo_l1_grid(f, xs, alpha, h):
    return np.array([caputo_l1_point(f, x, alpha, h) for x in np.asarray(xs, dtype=float)
])

```

## B.2 functions.py

```

from dataclasses import dataclass, field
from typing import Callable

import numpy as np
from scipy.special import gamma, rgamma

def _arr(x):
    return np.asarray(x, dtype=float)

@dataclass
class FuncSpec:
    key: str
    label: str
    f: Callable
    interval: tuple
    classical: dict = field(default_factory=dict)
    exact_frac: Callable = None

def power_ref(beta):
    def ref(x, alpha):
        x = _arr(x)
        return gamma(beta + 1.0) * rgamma(beta - alpha + 1.0) * np.power(x, beta - alpha)
    return ref

def safe_ln(x):

```

```

    x = _arr(x)
    out = np.full_like(x, np.nan)
    ok = x > 0
    out[ok] = np.log(x[ok])
    return out

def tan_fn(x):
    return np.tan(_arr(x))

REGISTRY = {}

def reg(spec):
    REGISTRY[spec.key] = spec

reg(FuncSpec("one", "1",
             lambda x: np.ones_like(_arr(x)),
             (1.0, 5.0),
             {1: lambda x: np.zeros_like(_arr(x)), 2: lambda x: np.zeros_like(_arr(x))}))

reg(FuncSpec("x", "x",
             lambda x: _arr(x),
             (1.0, 5.0),
             {1: lambda x: np.ones_like(_arr(x)), 2: lambda x: np.zeros_like(_arr(x))},
             power_ref(1.0)))

reg(FuncSpec("x2", "x^2",
             lambda x: _arr(x) ** 2,
             (1.0, 5.0),
             {1: lambda x: 2.0 * _arr(x), 2: lambda x: np.full_like(_arr(x), 2.0)},
             power_ref(2.0)))

reg(FuncSpec("x3", "x^3",
             lambda x: _arr(x) ** 3,
             (1.0, 5.0),
             {1: lambda x: 3.0 * _arr(x) ** 2, 2: lambda x: 6.0 * _arr(x)},
             power_ref(3.0)))

reg(FuncSpec("exp", "e^x",
             lambda x: np.exp(_arr(x)),
             (0.0, 3.0),
             {1: lambda x: np.exp(_arr(x)), 2: lambda x: np.exp(_arr(x))},
             lambda x, alpha: np.exp(_arr(x))))

reg(FuncSpec("sin", "sin x",

```

```

        lambda x: np.sin(_arr(x)),
        (0.0, 4.0 * np.pi),
        {1: lambda x: np.cos(_arr(x)), 2: lambda x: -np.sin(_arr(x))},
        lambda x, alpha: np.sin(_arr(x) + np.pi * alpha / 2.0)))

reg(FuncSpec("cos", "cos x",
            lambda x: np.cos(_arr(x)),
            (0.0, 4.0 * np.pi),
            {1: lambda x: -np.sin(_arr(x)), 2: lambda x: -np.cos(_arr(x))},
            lambda x, alpha: np.cos(_arr(x) + np.pi * alpha / 2.0)))

reg(FuncSpec("tan", "tan x",
            tan_fn,
            (0.0, 1.3),
            {1: lambda x: 1.0 / np.cos(_arr(x)) ** 2}))

reg(FuncSpec("ln", "ln x",
            safe_ln,
            (6.0, 10.0),
            {1: lambda x: 1.0 / _arr(x), 2: lambda x: -1.0 / _arr(x) ** 2}))

reg(FuncSpec("xsin", "x*sin x",
            lambda x: _arr(x) * np.sin(_arr(x)),
            (0.0, 4.0 * np.pi),
            {1: lambda x: np.sin(_arr(x)) + _arr(x) * np.cos(_arr(x)),
             2: lambda x: 2.0 * np.cos(_arr(x)) - _arr(x) * np.sin(_arr(x))}))

reg(FuncSpec("xexp", "x*e^x",
            lambda x: _arr(x) * np.exp(_arr(x)),
            (0.0, 3.0),
            {1: lambda x: np.exp(_arr(x)) * (1.0 + _arr(x)),
             2: lambda x: np.exp(_arr(x)) * (2.0 + _arr(x))}))

reg(FuncSpec("sin_plus_cos", "sin x + cos x",
            lambda x: np.sin(_arr(x)) + np.cos(_arr(x)),
            (0.0, 4.0 * np.pi),
            {1: lambda x: np.cos(_arr(x)) - np.sin(_arr(x)),
             2: lambda x: -np.sin(_arr(x)) - np.cos(_arr(x))},
            lambda x, alpha: (np.sin(_arr(x) + np.pi * alpha / 2.0)
                               + np.cos(_arr(x) + np.pi * alpha / 2.0))))

def get(key):
    if key not in REGISTRY:
        raise KeyError(f"нет функции '{key}'. Есть: {'', '.join(REGISTRY)}")
    return REGISTRY[key]

```

```
def names():
    return list(REGISTRY)
```

## B.3 experiments.py

```
import numpy as np
import pandas as pd
from scipy.special import gamma

import functions as fn
from fractional import (gl_grid, gl_point, gl_point_from_zero, gl_grid_from_zero,
                        gl_coefficients, richardson_point, richardson_from_zero,
                        coefficient_partial_sums, gl_shifted_point_from_zero,
                        caputo_l1_point)

def _successive_ratios(errors):
    ratios = [np.nan]
    for prev, cur in zip(errors, errors[1:]):
        ratios.append(prev / cur if prev > 0 and cur > 0 else np.nan)
    return ratios

def _method_errors(hs, exact, methods):
    table = {"h": list(hs)}
    for err_col, ratio_col, estimate in methods:
        errors = [abs(estimate(h) - exact) for h in hs]
        table[err_col] = errors
        table[ratio_col] = _successive_ratios(errors)
    return pd.DataFrame(table)

def table_values(func_key, xs, alphas, h, N):
    spec = fn.get(func_key)
    data = {"x": xs, "f(x)": spec.f(xs)}
    for a in alphas:
        data[f"D^{a:g}"] = gl_grid(spec.f, xs, a, h, N)
    return pd.DataFrame(data)

def table_values_from_zero(func_key, xs, alphas, h):
    spec = fn.get(func_key)
    data = {"x": xs, "f(x)": spec.f(xs)}
    for a in alphas:
        data[f"D^{a:g}"] = gl_grid_from_zero(spec.f, xs, a, h)
    return pd.DataFrame(data)
```

```

def master_table(h=0.01, N=500):
    alphas = [0.0, 0.5, 1.0, 1.5, 2.0]
    rows = []

    def add(label, x0, regime, values):
        row = {"функция": label, "x0": x0, "режим": regime}
        for a, v in zip(alphas, values):
            row[f"D^{a:g}"] = v
        rows.append(row)

    for key in ["one", "x", "x2", "x3"]:
        spec = fn.get(key)
        add(spec.label, 2.0, "a=0",
            [gl_point_from_zero(spec.f, 2.0, a, h) for a in alphas])

    for key in ["exp", "sin", "cos", "sin_plus_cos", "ln", "xsin", "xexp"]:
        spec = fn.get(key)
        x0 = 8.0 if key == "ln" else 1.0
        add(spec.label, x0, f"память L={N * h:g}",
            [gl_point(spec.f, x0, a, h, N) for a in alphas])

    spec = fn.get("tan")
    add(spec.label, 1.0, "память L=1",
        [gl_point(spec.f, 1.0, a, h, 100) for a in alphas])

    return pd.DataFrame(rows)

def regime_comparison(func_key, alpha, x, h, L, N_fixed):
    spec = fn.get(func_key)
    exact = float(spec.exact_frac(np.array([x]), alpha)[0])
    rows = [
        {"режим": "a=0 (N=x/h)", "N": int(round(x / h)),
         "значение": gl_point_from_zero(spec.f, x, alpha, h)},
        {"режим": f"память L={L:g}", "N": int(round(L / h)),
         "значение": gl_point(spec.f, x, alpha, h, int(round(L / h)))},
        {"режим": f"N={N_fixed}", "N": N_fixed,
         "значение": gl_point(spec.f, x, alpha, h, N_fixed)},
    ]
    df = pd.DataFrame(rows)
    df["эталон a=0"] = exact
    return df

def compare_classical(func_keys, xs, h, N, order):
    rows = []
    for key in func_keys:

```

```

    spec = fn.get(key)
    if order not in spec.classical:
        continue
    gl = gl_grid(spec.f, xs, float(order), h, N)
    exact = spec.classical[order](xs)
    for x, g, e in zip(xs, gl, exact):
        rows.append({
            "функция": spec.label,
            "x": x,
            "ГЛ": g,
            "точная": e,
            "абс. ошибка": abs(g - e),
            "отн. ошибка": abs(g - e) / max(1.0, abs(e)),
        })
    return pd.DataFrame(rows)

def h_influence(func_key, order, hs, xs, N):
    spec = fn.get(func_key)
    exact = spec.classical[order](xs)
    rows = []
    for h in hs:
        gl = gl_grid(spec.f, xs, float(order), h, N)
        err = np.abs(gl - exact)
        rows.append({"h": h, "N": N, "L=N*h": N * h,
                    "макс. ошибка": np.nanmax(err),
                    "сред. ошибка": np.nanmean(err)})
    return pd.DataFrame(rows)

def N_influence(func_key, alpha, Ns, h, xs):
    spec = fn.get(func_key)
    ref = spec.exact_frac(xs, alpha)
    rows = []
    for N in Ns:
        gl = gl_grid(spec.f, xs, alpha, h, N)
        err = np.abs(gl - ref)
        rows.append({"N": N, "L=N*h": N * h,
                    "макс. ошибка": np.nanmax(err),
                    "сред. ошибка": np.nanmean(err)})
    return pd.DataFrame(rows)

def convergence_integer(func_key, order, hs, x, N):
    spec = fn.get(func_key)
    exact = float(spec.classical[order](np.array([x]))[0])
    gls = [gl_point(spec.f, x, float(order), h, N) for h in hs]
    errs = [abs(gl - exact) for gl in gls]

```



```

return pd.DataFrame({
    "h": list(hs),
    "ГЛ": gls,
    "точная": exact,
    "ошибка": errs,
    "ошибка(h)/ошибка(h/2)": _successive_ratios(errs),
})

def convergence_power_frac(func_key, alpha, hs, x):
    spec = fn.get(func_key)
    exact = float(spec.exact_frac(np.array([x]), alpha)[0])
    gls = [gl_point_from_zero(spec.f, x, alpha, h) for h in hs]
    errs = [abs(gl - exact) for gl in gls]
    return pd.DataFrame({
        "h": list(hs),
        "N=x/h": [int(round(x / h)) for h in hs],
        "ГЛ": gls,
        "точная": exact,
        "ошибка": errs,
        "ошибка(h)/ошибка(h/2)": _successive_ratios(errs),
    })

def coefficient_table(alphas, kmax):
    data = {"k": np.arange(kmax + 1)}
    for a in alphas:
        data[f"alpha={a:g}"] = gl_coefficients(a, kmax)
    return pd.DataFrame(data)

def coefficient_decay(alphas, kmax):
    return {a: np.abs(gl_coefficients(a, kmax)) for a in alphas}

def rounding_floor(func_key, order, hs, x, N):
    spec = fn.get(func_key)
    exact = float(spec.classical[order](np.array([x]))[0])
    rows = []
    for h in hs:
        gl = gl_point(spec.f, x, float(order), h, N)
        rows.append({"h": h, "ГЛ": gl, "ошибка": abs(gl - exact)})
    return pd.DataFrame(rows)

def richardson_convergence(func_key, order, hs, x, N):
    spec = fn.get(func_key)
    exact = float(spec.classical[order](np.array([x]))[0])

```

```

return _method_errors(hs, exact, [
    ("ошибка (обычная)", "отнош. обычной",
     lambda h: gl_point(spec.f, x, float(order), h, N)),
    ("ошибка (Ричардсон)", "отнош. Ричардсона",
     lambda h: richardson_point(spec.f, x, float(order), h, N)),
])

def richardson_convergence_frac(func_key, alpha, hs, x):
    spec = fn.get(func_key)
    exact = float(spec.exact_frac(np.array([x]), alpha)[0])
    return _method_errors(hs, exact, [
        ("ошибка (обычная)", "отнош. обычной",
         lambda h: gl_point_from_zero(spec.f, x, alpha, h)),
        ("ошибка (Ричардсон)", "отнош. Ричардсона",
         lambda h: richardson_from_zero(spec.f, x, alpha, h)),
    ])

def truncation_order_N(func_key, alpha, Ns, h, xs):
    spec = fn.get(func_key)
    ref = spec.exact_frac(xs, alpha)
    rows = []
    prev_err = prev_N = None
    for N in Ns:
        gl = gl_grid(spec.f, xs, alpha, h, N)
        err = float(np.nanmax(np.abs(gl - ref)))
        slope = np.nan
        if prev_err is not None and err > 0 and prev_err > 0:
            slope = np.log(prev_err / err) / np.log(N / prev_N)
        rows.append({"N": N, "L=N*h": N * h, "макс. ошибка": err,
                     "оценка наклона": slope})
        prev_err, prev_N = err, N
    return pd.DataFrame(rows)

def coefficient_sum_table(alphas, ns, N):
    data = {"n": ns}
    for a in alphas:
        S = coefficient_partial_sums(a, N)
        data[f"alpha={a:g}"] = [S[n] for n in ns]
    return pd.DataFrame(data)

def atlas_matrix(func_key, alphas, xs, h, N):
    spec = fn.get(func_key)
    M = np.empty((len(alphas), len(xs)))
    for i, a in enumerate(alphas):

```

```

        M[i, :] = gl_grid(spec.f, xs, a, h, N)
    return M

def error_grid_hN(func_key, order, hs, Ns, xs):
    spec = fn.get(func_key)
    exact = spec.classical[order](xs)
    M = np.empty((len(hs), len(Ns)))
    for i, h in enumerate(hs):
        for j, N in enumerate(Ns):
            gl = gl_grid(spec.f, xs, float(order), h, N)
            M[i, j] = float(np.nanmax(np.abs(gl - exact)))
    return M

def error_grid_hN_frac(func_key, alpha, hs, Ns, xs):
    spec = fn.get(func_key)
    ref = spec.exact_frac(xs, alpha)
    M = np.empty((len(hs), len(Ns)))
    for i, h in enumerate(hs):
        for j, N in enumerate(Ns):
            gl = gl_grid(spec.f, xs, alpha, h, N)
            M[i, j] = float(np.nanmax(np.abs(gl - ref)))
    return M

def shifted_vs_standard(func_key, alpha, hs, x):
    spec = fn.get(func_key)
    exact = float(spec.exact_frac(np.array([x]), alpha)[0])
    return _method_errors(hs, exact, [
        ("ошибка ГЛ (p=0)", "отнош. ГЛ",
         lambda h: gl_point_from_zero(spec.f, x, alpha, h)),
        ("ошибка сдвиг (p=1)", "отнош. сдвиг",
         lambda h: gl_shifted_point_from_zero(spec.f, x, alpha, h)),
    ])

def gl_vs_caputo(func_key, alpha, xs, h):
    spec = fn.get(func_key)
    f0 = float(spec.f(np.array([0.0]))[0])
    glv = gl_grid_from_zero(spec.f, xs, alpha, h)
    rows = []
    for x, g in zip(xs, glv):
        cap = caputo_l1_point(spec.f, x, alpha, h)
        corr = f0 * x ** (-alpha) / gamma(1.0 - alpha)
        rows.append({"x": x, "ГЛ (a=0)": g, "Капуто (L1)": cap,
                     "ГЛ - Капуто": g - cap,
                     "f(0)*x^{-a}/Г(1-a)": corr})

```

```
return pd.DataFrame(rows)
```

## B.4 tables.py

```
import os

import numpy as np
import pandas as pd

import experiments as ex

TABLES = os.path.join(os.path.dirname(__file__), "tables")

H_BASE = 0.01
N_BASE = 500

def build_all():
    os.makedirs(TABLES, exist_ok=True)
    out = {}

    out["master_table"] = ex.master_table(H_BASE, N_BASE)

    xs = np.linspace(0, 2 * np.pi, 9)
    out["sin_values"] = ex.table_values(
        "sin", xs, [0.0, 0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0],
        H_BASE, N_BASE)

    xs2 = np.linspace(1, 5, 9)
    out["x2_values"] = ex.table_values_from_zero(
        "x2", xs2, [0.0, 0.5, 1.0, 1.5, 2.0], H_BASE)

    xs3 = np.linspace(0.5, 5, 6)
    out["compare_alpha1"] = ex.compare_classical(
        ["x2", "exp", "sin", "cos"], xs3, H_BASE, N_BASE, order=1)

    xs4 = np.linspace(0.5, 2 * np.pi, 200)
    out["h_influence"] = ex.h_influence(
        "sin", 1, [0.1, 0.05, 0.02, 0.01, 0.005, 0.001], xs4, N_BASE)

    xs5 = np.linspace(np.pi, 3 * np.pi, 200)
    out["N_influence"] = ex.N_influence(
        "sin", 0.5, [5, 20, 50, 100, 300, 500, 1000], H_BASE, xs5)

    out["convergence_int"] = ex.convergence_integer(
        "sin", 1, [0.2, 0.1, 0.05, 0.025, 0.0125, 0.00625], x=1.0, N=50)
```

```

out["convergence_frac"] = ex.convergence_power_frac(
    "x2", 0.5, [0.04, 0.02, 0.01, 0.005, 0.0025, 0.00125], x=2.0)

out["coefficients"] = ex.coefficient_table([0.0, 0.5, 1.0, 1.5, 2.0], kmax=6)

out["rounding_floor"] = ex.rounding_floor(
    "sin", 1, [0.1, 0.01, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8, 1e-9,
               1e-10, 1e-11], x=1.0, N=2)

out["richardson"] = ex.richardson_convergence(
    "sin", 1, [0.2, 0.1, 0.05, 0.025, 0.0125, 0.00625], x=1.0, N=2)

out["richardson_frac"] = ex.richardson_convergence_frac(
    "x2", 0.5, [0.04, 0.02, 0.01, 0.005, 0.0025, 0.00125], x=2.0)

xs6 = np.linspace(np.pi, 3 * np.pi, 200)
out["truncation_N"] = ex.truncation_order_N(
    "sin", 0.5, [5, 20, 50, 100, 300, 500, 1000, 2000], H_BASE, xs6)

out["coeff_sums"] = ex.coefficient_sum_table(
    [0.25, 0.5, 0.75], ns=[1, 5, 10, 50, 100, 500], N=500)

out["x3_values"] = ex.table_values_from_zero(
    "x3", np.linspace(1, 4, 7), [0.0, 0.5, 1.0, 1.5, 2.0], H_BASE)

out["xsin_values"] = ex.table_values(
    "xsin", np.linspace(0, 2 * np.pi, 9), [0.0, 0.5, 1.0, 2.0], H_BASE, N_BASE)

out["regime_compare"] = ex.regime_comparison(
    "x2", 0.5, x=2.0, h=0.01, L=5.0, N_fixed=50)

out["shifted_vs_standard"] = ex.shifted_vs_standard(
    "x2", 0.5, [0.04, 0.02, 0.01, 0.005, 0.0025], x=2.0)

out["gl_vs_caputo"] = ex.gl_vs_caputo(
    "exp", 0.5, np.linspace(0.5, 2.5, 6), h=0.005)

for name, df in out.items():
    df.to_csv(os.path.join(TABLES, f"{name}.csv"), index=False,
              encoding="utf-8-sig")
with pd.ExcelWriter(os.path.join(TABLES, "tables.xlsx")) as writer:
    for name, df in out.items():
        df.to_excel(writer, sheet_name=name[:31], index=False)
return out

if __name__ == "__main__":
    for name, df in build_all().items():

```

```

print(f"\n=== {name} ===")
print(df.to_string(index=False))

```

## B.5 plots.py

```

import os

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from scipy.special import gamma

import functions as fn
from fractional import (gl_grid, gl_grid_right, gl_point, gl_point_from_zero,
                        gl_grid_from_zero, gl_coefficients,
                        coefficient_partial_sums, gl_shifted_point_from_zero,
                        caputo_l1_grid)
import experiments as ex

plt.rcParams.update({
    "font.family": "DejaVu Sans",
    "mathtext.fontset": "dejavusans",
    "figure.dpi": 130,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "savefig.bbox": "tight",
})

GRAPHS = os.path.join(os.path.dirname(__file__), "graphs")

def _save(fig, name):
    os.makedirs(GRAPHS, exist_ok=True)
    path = os.path.join(GRAPHS, name)
    fig.savefig(path)
    plt.close(fig)
    return path

def sin_orders(h=0.01, N=500):
    spec = fn.get("sin")
    xs = np.linspace(0, 4 * np.pi, 700)
    fig, ax = plt.subplots(figsize=(8, 4.5))
    for a in [0.0, 0.5, 1.0, 1.5, 2.0]:
        ax.plot(xs, gl_grid(spec.f, xs, a, h, N), label=fr"$\alpha = {a:g}$")

```

```

ax.plot(xs, np.cos(xs), "k--", lw=1, alpha=0.6, label=r"$\cos x$ ($\alpha=1$)")
ax.plot(xs, -np.sin(xs), "k:", lw=1, alpha=0.6, label=r"$-\sin x$ ($\alpha=2$)")
ax.set_xlabel("x"); ax.set_ylabel("значение")
ax.set_title(r"Дробные производные $\sin x$ при разных порядках $\alpha$")
ax.legend(fontsize=8, ncol=2)
return _save(fig, "sin_orders.png")

def alpha1_check(h=0.01, N=500):
    spec = fn.get("sin")
    xs = np.linspace(0, 2 * np.pi, 400)
    gl = gl_grid(spec.f, xs, 1.0, h, N)
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(xs, gl, label=r"$D^1$ (ГЛ)", lw=2)
    ax.plot(xs, np.cos(xs), "r--", label=r"$\cos x$")
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r"$D^1 \sin x$ против $\cos x$")
    ax.legend(fontsize=9)
    return _save(fig, "alpha1_check.png")

def h_influence(N=500):
    spec = fn.get("sin")
    xs = np.linspace(0, 2 * np.pi, 500)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.plot(xs, np.cos(xs), "k-", lw=2, label=r"$\cos x$")
    for h, style in [(0.5, "C0--"), (0.2, "C1-."), (0.02, "C2:")]:
        ax.plot(xs, gl_grid(spec.f, xs, 1.0, h, N), style,
                label=fr"$h={h:g}$")
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r"Влияние шага $h$ на $D^1 \sin x$")
    ax.legend(fontsize=9)
    return _save(fig, "h_influence.png")

def N_influence(h=0.01):
    spec = fn.get("sin")
    xs = np.linspace(np.pi, 3 * np.pi, 400)
    ref = spec.exact_frac(xs, 0.5)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.plot(xs, ref, "k-", lw=2, label=r"$\sin(x+\pi/4)$")
    for N, style in [(5, "C0--"), (20, "C1-."), (200, "C2:")]:
        ax.plot(xs, gl_grid(spec.f, xs, 0.5, h, N), style, label=fr"$N={N}$")
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r"Влияние числа членов $N$ на $D^{0.5} \sin x$")
    ax.legend(fontsize=9)
    return _save(fig, "N_influence.png")

```

```

def coeff_decay(kmax=500):
    fig, ax = plt.subplots(figsize=(7.5, 4.5))
    k = np.arange(1, kmax + 1)
    for a in [0.5, 1.5]:
        c = np.abs(gl_coefficients(a, kmax))[1:]
        ax.loglog(k, c, label=fr"$\alpha={a:g}$ (дробный)")
    c1 = np.abs(gl_coefficients(1.0, kmax))[1:]
    nz = c1 > 0
    ax.loglog(k[nz], c1[nz], "o", ms=7, label=r"$\alpha=1$ (один член)")
    c_inf = 1.0 / abs(gamma(-0.5))
    ax.loglog(k, c_inf * k ** (-1.5), "k--", lw=1, alpha=0.6,
              label=r"асимптотика $k^{-(\alpha+1)}/|\Gamma(-\alpha)|$, $\alpha=0.5$")
    ax.set_xlabel("k"); ax.set_ylabel(r"$|c_k|$")
    ax.set_title(r"Затухание коэффициентов $c_k$")
    ax.legend(fontsize=9)
    return _save(fig, "coeff_decay.png")

def power_frac():
    spec = fn.get("x2")
    xs = np.linspace(0.2, 5, 300)
    h = 0.01
    half = gl_grid_from_zero(spec.f, xs, 0.5, h)
    ref_half = spec.exact_frac(xs, 0.5)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.plot(xs, spec.f(xs), label=r"$x^2$ ($\alpha=0$)")
    ax.plot(xs, half, "C1", lw=2, label=r"$D^{0.5}$ (ГЛ)")
    ax.plot(xs, ref_half, "C1--", lw=1, label=r"$D^{0.5}$ (через $\Gamma$)")
    ax.plot(xs, 2 * xs, "C2:", label=r"$2x$ ($\alpha=1$)")
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r"Дробная производная $x^2$")
    ax.legend(fontsize=9)
    return _save(fig, "power_frac.png")

def convergence():
    hs = np.array([0.2, 0.1, 0.05, 0.025, 0.0125, 0.00625])
    spec_sin = fn.get("sin")
    err_sin = [abs(gl_point(spec_sin.f, 1.0, 1.0, h, 50) - np.cos(1.0)) for h in hs]
    spec_x2 = fn.get("x2")
    ref = float(spec_x2.exact_frac(np.array([2.0]), 0.5)[0])
    err_x2 = [abs(gl_point_from_zero(spec_x2.f, 2.0, 0.5, h) - ref) for h in hs]
    fig, ax = plt.subplots(figsize=(7.5, 4.5))
    ax.loglog(hs, err_sin, "o-", label=r"$\alpha=1$, $\sin x$, $x=1$")
    ax.loglog(hs, err_x2, "s-", label=r"$\alpha=0.5$, $x^2$, $x=2$")
    ax.loglog(hs, hs * (err_sin[0] / hs[0]), "k--", lw=1, alpha=0.6,
              label=r"$O(h)$")

```



```

ax.set_xlabel(r"шаг  $h$ "); ax.set_ylabel("абсолютная ошибка")
ax.set_title(r"Сходимость метода: ошибка  $\sim O(h)$ ")
ax.legend(fontsize=9)
return _save(fig, "convergence.png")

def exp_orders(h=0.01, N=500):
    spec = fn.get("exp")
    xs = np.linspace(0, 3, 300)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    for a in [0.0, 0.5, 1.0, 1.5, 2.0]:
        ax.plot(xs, gl_grid(spec.f, xs, a, h, N), label=fr" $\alpha={a:g}$ ")
    ax.plot(xs, np.exp(xs), "k--", lw=1, alpha=0.5, label=r" $e^x$ ")
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r"Дробные производные  $e^x$ ")
    ax.legend(fontsize=8, ncol=2)
    return _save(fig, "exp_orders.png")

def ln_orders(h=0.01, N=500):
    spec = fn.get("ln")
    xs = np.linspace(6, 10, 300)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.plot(xs, spec.f(xs), label=r" $\ln x$  ( $\alpha=0$ )")
    for a in [0.5, 1.0]:
        ax.plot(xs, gl_grid(spec.f, xs, a, h, N), label=fr" $\alpha={a:g}$ ")
    ax.plot(xs, 1.0 / xs, "k--", lw=1, alpha=0.6, label=r" $1/x$  ( $\alpha=1$ )")
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r" $\ln x$  на  $[6,10]$  при условии  $x-Nh>0$ ")
    ax.legend(fontsize=9)
    return _save(fig, "ln_orders.png")

def left_vs_right(h=0.01, N=500):
    spec = fn.get("sin")
    xs = np.linspace(np.pi, 3 * np.pi, 400)
    left = gl_grid(spec.f, xs, 0.5, h, N)
    right = gl_grid_right(spec.f, xs, 0.5, h, N)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.plot(xs, np.sin(xs), "k-", lw=1, alpha=0.5, label=r" $\sin x$ ")
    ax.plot(xs, left, "C0", lw=2, label=r"левая  $D^{0.5}$ ")
    ax.plot(xs, right, "C3", lw=2, label=r"правая  $D^{0.5}$ ")
    ax.plot(xs, np.sin(xs + np.pi / 4), "C0--", lw=1, label=r" $\sin(x+\pi/4)$ ")
    ax.plot(xs, np.sin(xs - np.pi / 4), "C3--", lw=1, label=r" $\sin(x-\pi/4)$ ")
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r"Левая и правая  $D^{0.5} \sin x$ ")
    ax.legend(fontsize=8, ncol=2)
    return _save(fig, "left_vs_right.png")

```

```

def atlas(h=0.01, N=500):
    spec = fn.get("sin")
    alphas = np.linspace(0, 2, 81)
    xs = np.linspace(0, 4 * np.pi, 300)
    M = ex.atlas_matrix("sin", alphas, xs, h, N)
    fig, ax = plt.subplots(figsize=(8.5, 4.5))
    im = ax.imshow(M, aspect="auto", origin="lower", cmap="RdBu_r",
                   extent=[xs[0], xs[-1], alphas[0], alphas[-1]],
                   vmin=-1.2, vmax=1.2)
    for a in [1.0, 2.0]:
        ax.axhline(a, color="k", lw=0.6, ls=":")
    ax.set_xlabel("x"); ax.set_ylabel(r"порядок  $\alpha$ ")
    ax.set_title(r"Карта значений  $D^{\alpha} \sin x$  (цвет - значение)")
    fig.colorbar(im, ax=ax, label="значение")
    ax.grid(False)
    return _save(fig, "atlas.png")

def rounding_floor():
    hs = np.logspace(-1, -12, 30)
    df = ex.rounding_floor("sin", 1, list(hs), x=1.0, N=2)
    fig, ax = plt.subplots(figsize=(7.5, 4.5))
    ax.loglog(df["h"], df["ошибка"], "o-", ms=4, label=r"ошибка  $D^{-1} \sin x$ ,  $x=1$ ")
    ax.loglog(hs, 0.42 * hs, "k--", lw=1, alpha=0.6, label=r"дискретизация  $\sim h/2$ ")
    ax.loglog(hs, 1e-16 / hs, "r:", lw=1, alpha=0.7, label=r"округление  $\sim \epsilon/h$ ")
    ax.set_xlabel(r"шаг  $h$ "); ax.set_ylabel("абсолютная ошибка")
    ax.set_title(r"Порог округления: при очень малом  $h$  ошибка снова растёт")
    ax.legend(fontsize=9)
    return _save(fig, "rounding_floor.png")

def richardson():
    hs = np.array([0.2, 0.1, 0.05, 0.025, 0.0125, 0.00625])
    df = ex.richardson_convergence("sin", 1, list(hs), x=1.0, N=2)
    fig, ax = plt.subplots(figsize=(7.5, 4.5))
    ax.loglog(hs, df["ошибка (обычная)"], "o-", label=r"обычная схема,  $O(h)$ ")
    ax.loglog(hs, df["ошибка (Ричардсон)"], "s-", label=r"Ричардсон,  $O(h^2)$ ")
    ax.loglog(hs, hs * (df["ошибка (обычная)"].iloc[0] / hs[0]), "k--", lw=1,
              alpha=0.5, label=r"наклон 1")
    ax.loglog(hs, hs ** 2 * (df["ошибка (Ричардсон)"].iloc[0] / hs[0] ** 2),
              "k:", lw=1, alpha=0.5, label=r"наклон 2")
    ax.set_xlabel(r"шаг  $h$ "); ax.set_ylabel("абсолютная ошибка")
    ax.set_title(r"Экстраполяция Ричардсона,  $\alpha=1$ ")
    ax.legend(fontsize=9)
    return _save(fig, "richardson.png")

```

```

def richardson_frac():
    hs = np.array([0.04, 0.02, 0.01, 0.005, 0.0025, 0.00125])
    df = ex.richardson_convergence_frac("x2", 0.5, list(hs), x=2.0)
    fig, ax = plt.subplots(figsize=(7.5, 4.5))
    ax.loglog(hs, df["ошибка (обычная)"], "o-", label=r"обычная схема,  $O(h)$ ")
    ax.loglog(hs, df["ошибка (Ричардсон)"], "s-", label=r"Ричардсон,  $O(h^2)$ ")
    ax.loglog(hs, hs * (df["ошибка (обычная)"].iloc[0] / hs[0]), "k--", lw=1,
               alpha=0.5, label=r"наклон 1")
    ax.loglog(hs, hs ** 2 * (df["ошибка (Ричардсон)"].iloc[0] / hs[0] ** 2),
               "k:", lw=1, alpha=0.5, label=r"наклон 2")
    ax.set_xlabel(r"шаг  $h$ "); ax.set_ylabel("абсолютная ошибка")
    ax.set_title(r"Экстраполяция Ричардсона для  $D^{0.5} x^2$ ,  $x=2$ ")
    ax.legend(fontsize=9)
    return _save(fig, "richardson_frac.png")

def N_truncation():
    spec = fn.get("sin")
    xs = np.linspace(np.pi, 3 * np.pi, 200)
    Ns = np.array([5, 10, 20, 50, 100, 200, 300, 500, 1000, 2000])
    ref = spec.exact_frac(xs, 0.5)
    errs = [float(np.nanmax(np.abs(gl_grid(spec.f, xs, 0.5, 0.01, N) - ref)))
             for N in Ns]
    fig, ax = plt.subplots(figsize=(7.5, 4.5))
    ax.loglog(Ns, errs, "o-", label=r"макс. ошибка  $D^{0.5} \sin x$ ")
    ax.loglog(Ns, errs[0] * (Ns / Ns[0]) ** (-0.5), "k--", lw=1, alpha=0.6,
               label=r"оценка усечения  $N^{-\alpha}$ ,  $\alpha=0.5$ ")
    ax.set_xlabel(r"число членов  $N$ "); ax.set_ylabel("макс. ошибка")
    ax.set_title(r"Усечение истории: ошибка убывает быстрее  $N^{-\alpha}$ ")
    ax.legend(fontsize=9)
    return _save(fig, "N_truncation.png")

def coeff_partial_sums(N=500):
    fig, ax = plt.subplots(figsize=(7.5, 4.2))
    n = np.arange(N + 1)
    for a in [0.25, 0.5, 0.75]:
        ax.plot(n, coefficient_partial_sums(a, N), label=fr" $\alpha={a:g}$ ")
    ax.axhline(0, color="k", lw=0.6)
    ax.set_xlabel("n"); ax.set_ylabel(r" $S_n$ ")
    ax.set_title(r"Частичные суммы коэффициентов  $S_n$   $\rightarrow 0$ ")
    ax.legend(fontsize=9)
    return _save(fig, "coeff_sums.png")

def error_heatmap():

```

```

hs = np.array([0.2, 0.1, 0.05, 0.02, 0.01, 0.005])
Ns = np.array([10, 20, 50, 100, 200, 500, 1000])
xs = np.linspace(np.pi, 3 * np.pi, 120)
M = ex.error_grid_hN("sin", 1, hs, Ns, xs)
fig, ax = plt.subplots(figsize=(8, 4.5))
im = ax.imshow(np.log10(M), aspect="auto", origin="upper", cmap="viridis",
                extent=[0, len(Ns), 0, len(hs)])
ax.set_xticks(np.arange(len(Ns)) + 0.5); ax.set_xticklabels(Ns)
ax.set_yticks(np.arange(len(hs)) + 0.5); ax.set_yticklabels(hs[::-1])
ax.set_xlabel(r"$N$"); ax.set_ylabel(r"$h$")
ax.set_title(r"$\log_{10}$ макс. ошибки $D^1 \sin x$ по сетке $(h, N)$")
fig.colorbar(im, ax=ax, label=r"$\log_{10}$ ошибки")
ax.grid(False)
return _save(fig, "error_heatmap.png")

def error_heatmap_frac():
    hs = np.array([0.2, 0.1, 0.05, 0.02, 0.01, 0.005])
    Ns = np.array([10, 20, 50, 100, 200, 500, 1000])
    xs = np.linspace(np.pi, 3 * np.pi, 120)
    M = ex.error_grid_hN_frac("sin", 0.5, hs, Ns, xs)
    fig, ax = plt.subplots(figsize=(8, 4.5))
    im = ax.imshow(np.log10(M), aspect="auto", origin="upper", cmap="viridis",
                    extent=[0, len(Ns), 0, len(hs)])
    ax.set_xticks(np.arange(len(Ns)) + 0.5); ax.set_xticklabels(Ns)
    ax.set_yticks(np.arange(len(hs)) + 0.5); ax.set_yticklabels(hs[::-1])
    ax.set_xlabel(r"$N$"); ax.set_ylabel(r"$h$")
    ax.set_title(r"$\log_{10}$ макс. ошибки $D^{0.5} \sin x$ по сетке $(h, N)$")
    fig.colorbar(im, ax=ax, label=r"$\log_{10}$ ошибки")
    ax.grid(False)
    return _save(fig, "error_heatmap_frac.png")

def tan_instability(h=0.01, N=100):
    spec = fn.get("tan")
    xs = np.linspace(0.2, 1.3, 300)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.plot(xs, spec.f(xs), label=r"$\tan x$")
    ax.plot(xs, gl_grid(spec.f, xs, 0.5, h, N), label=r"$D^{0.5}$")
    ax.plot(xs, gl_grid(spec.f, xs, 1.0, h, N), label=r"$D^1$ (ГЛ)")
    ax.plot(xs, 1.0 / np.cos(xs) ** 2, "k--", lw=1, alpha=0.6, label=r"$1/\cos^2 x$")
    ax.set_ylim(0, 15)
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r"$\tan x$ на $[0.2, 1.3]$ около $\pi/2$")
    ax.legend(fontsize=9)
    return _save(fig, "tan_instability.png")

```

```

def x3_orders(h=0.01):
    spec = fn.get("x3")
    xs = np.linspace(0.3, 4, 300)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.plot(xs, spec.f(xs), label=r"$x^3$ ($\alpha=0$)")
    for a in [0.5, 1.0, 1.5, 2.0]:
        y = gl_grid_from_zero(spec.f, xs, a, h)
        ax.plot(xs, y, label=fr"$\alpha={a:g}$")
    ax.plot(xs, 6 * xs, "k--", lw=1, alpha=0.5, label=r"$6x$ ($\alpha=2$)")
    ax.set_ylim(-5, 65)
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r"Дробные производные $x^3$")
    ax.legend(fontsize=8, ncol=2)
    return _save(fig, "x3_orders.png")

def shifted_convergence(x=2.0):
    spec = fn.get("x2")
    exact = float(spec.exact_frac(np.array([x]), 0.5)[0])
    hs = np.array([0.04, 0.02, 0.01, 0.005, 0.0025])
    e_std = np.array([abs(gl_point_from_zero(spec.f, x, 0.5, h) - exact) for h in hs])
    e_sh = np.array([abs(gl_shifted_point_from_zero(spec.f, x, 0.5, h) - exact) for h in hs])
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.loglog(hs, e_std, "o-", label=r"стандартная ГЛ ($p=0$)")
    ax.loglog(hs, e_sh, "s-", label=r"сдвинутая ГЛ ($p=1$)")
    ax.loglog(hs, hs * (e_std[0] / hs[0]), "k--", lw=1, alpha=0.5, label=r"наклон $1$")
    ax.set_xlabel("шаг h"); ax.set_ylabel("ошибка")
    ax.set_title(r"Стандартная и сдвинутая ГЛ для $D^{0.5} x^2$")
    ax.legend(fontsize=9)
    return _save(fig, "shifted_convergence.png")

def gl_vs_caputo(alpha=0.5, h=0.005):
    spec = fn.get("exp")
    xs = np.linspace(0.3, 2.5, 200)
    gl = gl_grid_from_zero(spec.f, xs, alpha, h)
    cap = caputo_l1_grid(spec.f, xs, alpha, h)
    corr = xs ** (-alpha) / gamma(1.0 - alpha)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.plot(xs, gl, label=r"Грюнвальд--Летников ($a=0$)")
    ax.plot(xs, cap, label=r"Капуто (L1)")
    ax.plot(xs, cap + corr, "k--", lw=1, alpha=0.6,
            label=r"Капуто $+ \backslash, x^{-\alpha} / \Gamma(1-\alpha)$")
    ax.set_xlabel("x"); ax.set_ylabel("значение")
    ax.set_title(r"Грюнвальд--Летников и Капуто для $D^{0.5} e^x$")
    ax.legend(fontsize=9)
    return _save(fig, "gl_vs_caputo.png")

```

```

def build_all():
    return [
        sin_orders(), alpha1_check(), h_influence(), N_influence(),
        coeff_decay(), power_frac(), convergence(), exp_orders(), ln_orders(),
        left_vs_right(), atlas(), rounding_floor(), richardson(),
        richardson_frac(), N_truncation(), coeff_partial_sums(),
        error_heatmap(), error_heatmap_frac(), tan_instability(), x3_orders(),
        shifted_convergence(), gl_vs_caputo(),
    ]

if __name__ == "__main__":
    for p in build_all():
        print("сохранён", p)

```

## Б.6 main.py

```

import argparse

import numpy as np
import pandas as pd

import functions as fn
from fractional import gl_point, gl_grid

def run_all():
    import tables
    import plots
    tabs = tables.build_all()
    print("Таблицы сохранены в tables/ (CSV + tables.xlsx):")
    for name in tabs:
        print("  -", name)
    for p in plots.build_all():
        print("график:", p)

def main():
    p = argparse.ArgumentParser(description="Дробная производная Грюнвальда-Летникова")
    p.add_argument("--all", action="store_true")
    p.add_argument("--function", default="sin", help=f"одна из: {' , '.join(fn.names())}")
    p.add_argument("--alpha", type=float, default=0.5)
    p.add_argument("--h", type=float, default=0.01)
    p.add_argument("--n", type=int, default=500)
    p.add_argument("--x", type=float, default=None)

```

```

p.add_argument("--xmin", type=float, default=None)
p.add_argument("--xmax", type=float, default=None)
p.add_argument("--points", type=int, default=9)
p.add_argument("--csv", default=None)
args = p.parse_args()

if args.all:
    run_all()
    return

spec = fn.get(args.function)
if args.x is not None:
    value = gl_point(spec.f, args.x, args.alpha, args.h, args.n)
    print(f"D^{args.alpha:g} [{spec.label}] (x={args.x:g}) = {value:.6f}")
    if args.csv:
        df = pd.DataFrame({"x": [args.x], "f(x)": [float(spec.f(np.array([args.x])))
[0]]},
                           f"D^{args.alpha:g}": [value]})
        df.to_csv(args.csv, index=False, encoding="utf-8-sig")
        print("сохранено в", args.csv)
    return

xmin = args.xmin if args.xmin is not None else spec.interval[0]
xmax = args.xmax if args.xmax is not None else spec.interval[1]
xs = np.linspace(xmin, xmax, args.points)
y = gl_grid(spec.f, xs, args.alpha, args.h, args.n)
df = pd.DataFrame({"x": xs, "f(x)": spec.f(xs), f"D^{args.alpha:g}": y})
print(df.to_string(index=False))
if args.csv:
    df.to_csv(args.csv, index=False, encoding="utf-8-sig")
    print("сохранено в", args.csv)

if __name__ == "__main__":
    main()

```

## Б.7 web/src/domain/fractional.ts

```

import type { MathFn } from "../types";

function grunwaldWeights(order: number, terms: number): number[] {
    const weights: number[] = [1];
    for (let k = 1; k <= terms; k += 1) {
        weights.push((weights[k - 1] * (k - 1 - order)) / k);
    }
    return weights;
}

```

```

function makeLeftDerivative(
  f: MathFn,
  order: number,
  step: number,
  terms: number,
  shift: number,
): (x: number) => number {
  const weights = grunwaldWeights(order, terms);
  const scale = step ** order;

  return (x) => {
    let total = 0;
    for (let k = 0; k < weights.length; k += 1) {
      const sample = f(x - (k - shift) * step);
      if (!Number.isFinite(sample)) {
        return Number.NaN;
      }
      total += weights[k] * sample;
    }
    return total / scale;
  };
}

export function glGrid(f: MathFn, xs: number[], alpha: number, h: number, n: number):
  number[] {
  return xs.map(makeLeftDerivative(f, alpha, h, n, 0));
}

export function glGridFromZero(f: MathFn, xs: number[], alpha: number, h: number): number
  [] {
  const positives = xs.filter((x) => x > 0);
  const nmax = positives.length > 0 ? Math.floor(Math.max(...positives) / h + 1e-12) : 0;
  const weights = grunwaldWeights(alpha, nmax);

  return xs.map((x) => {
    const n = Math.floor(x / h + 1e-12);
    if (n < 0) {
      return Number.NaN;
    }
    let total = 0;
    for (let k = 0; k <= n; k += 1) {
      const sample = f(x - k * h);
      if (!Number.isFinite(sample)) {
        return Number.NaN;
      }
      total += weights[k] * sample;
    }
  });
}

```



```

    return total / h ** alpha;
  });
}

export function glShiftedGrid(
  f: MathFn,
  xs: number[],
  alpha: number,
  h: number,
  n: number,
  p = 1,
): number[] {
  return xs.map(makeLeftDerivative(f, alpha, h, n, p));
}

export function historyLength(h: number, n: number): number {
  return h * n;
}

function gamma(z: number): number {
  let reduction = 1;
  let w = z;
  while (w < 12) {
    reduction *= w;
    w += 1;
  }
  const inv = 1 / w;
  const stirling =
    (w - 0.5) * Math.log(w) -
    w +
    0.5 * Math.log(2 * Math.PI) +
    inv / 12 -
    inv ** 3 / 360 +
    inv ** 5 / 1260;
  return Math.exp(stirling) / reduction;
}

export function caputoL1Grid(f: MathFn, xs: number[], alpha: number, h: number): number[]
{
  if (alpha <= 0 || alpha >= 1) {
    return xs.map(() => Number.NaN);
  }

  const stepCounts = xs.map((x) => (x > 0 ? Math.floor(x / h + 1e-12) : 0));
  const maxSteps = stepCounts.reduce((acc, value) => Math.max(acc, value), 0);

  const samples = Array.from({ length: maxSteps + 1 }, (_, j) => f(j * h));
  const weights = Array.from({ length: maxSteps + 1 }, (_, j) =>

```

```

    j >= 1 ? j ** (1 - alpha) - (j - 1) ** (1 - alpha) : 0,
  );
  const firstBad = samples.findIndex((value) => !Number.isFinite(value));
  const scale = gamma(2 - alpha) * h ** alpha;

  return xs.map((_, index) => {
    const steps = stepCounts[index];
    if (steps < 1 || (firstBad !== -1 && firstBad <= steps)) {
      return Number.NaN;
    }
    let total = 0;
    for (let j = 1; j <= steps; j += 1) {
      total += weights[j] * (samples[steps - j + 1] - samples[steps - j]);
    }
    return total / scale;
  });
}

```

## B.8 web/src/domain/derivativeModel.ts

```

import { glGrid, glGridFromZero, glShiftedGrid, caputoL1Grid } from "../fractional";
import { formatOrder } from "../format";
import { linspace } from "../grid";
import { POINT_COUNT, TABLE_STEP } from "../options";
import type { CalculatorSettings, ClassicalOrder, Scheme, TableRow } from "../types";

const SCHEME_SHORT: Record<Scheme, string> = {
  standard: "ГЛ",
  fromZero: "ГЛ a=0",
  shifted: "сдвиг",
  caputo: "Капуто",
};

function computeDerivative(settings: CalculatorSettings, xs: number[]): number[] {
  switch (settings.scheme) {
    case "fromZero":
      return glGridFromZero(settings.spec.f, xs, settings.alpha, settings.h);
    case "shifted":
      return glShiftedGrid(settings.spec.f, xs, settings.alpha, settings.h, settings.n, 1);
    case "caputo":
      return caputoL1Grid(settings.spec.f, xs, settings.alpha, settings.h);
    default:
      return glGrid(settings.spec.f, xs, settings.alpha, settings.h, settings.n);
  }
}

```

```

export type ChartSeries = {
  name: string;
  color: string;
  dashed?: boolean;
  strokeWidth?: number;
  values: number[];
};

export type DerivativeModel = {
  xs: number[];
  rows: TableRow[];
  series: ChartSeries[];
  classicalOrder: ClassicalOrder | null;
  error: {
    value: number | null;
    sampleCount: number;
  };
};

export function buildDerivativeModel(settings: CalculatorSettings): DerivativeModel {
  const xmin = Math.min(settings.xmin, settings.xmax);
  const xmax = Math.max(settings.xmin, settings.xmax);
  const xs = linspace(xmin, xmax, POINT_COUNT);
  const functionValues = xs.map(settings.spec.f);
  const derivativeValues = computeDerivative(settings, xs);
  const classicalOrder = getClassicalOrder(settings.alpha);
  const exactFn =
    classicalOrder === null
      ? undefined
      : classicalOrder === 0
        ? settings.spec.f
        : settings.spec.classical[classicalOrder];
  const exactValues = exactFn ? xs.map(exactFn) : [];
  const rows = makeRows(xs, functionValues, derivativeValues, exactValues);
  const series = makeSeries(settings, functionValues, derivativeValues, exactValues,
    classicalOrder);

  return {
    xs,
    rows,
    series,
    classicalOrder,
    error: calculateAbsoluteError(derivativeValues, exactValues),
  };
}

export function makeCsvRows(model: DerivativeModel): Array<Record<string, string | number
  | undefined>> {

```

```

return model.rows.map((row) => ({
  x: row.x,
  "f(x)": row.fx,
  "D $\alpha$  f(x)": row.derivative,
  "эталон": row.exact,
}));
}

function getClassicalOrder(alpha: number): ClassicalOrder | null {
  const rounded = Math.round(alpha);
  if (Math.abs(alpha - rounded) > 1e-9) {
    return null;
  }

  if (rounded === 0 || rounded === 1 || rounded === 2) {
    return rounded;
  }

  return null;
}

function calculateAbsoluteError(values: number[], exactValues: number[]) {
  let max = 0;
  let sampleCount = 0;

  values.forEach((value, index) => {
    const exact = exactValues[index];
    if (!Number.isFinite(value) || !Number.isFinite(exact)) {
      return;
    }

    max = Math.max(max, Math.abs(value - exact));
    sampleCount += 1;
  });

  return {
    value: sampleCount === 0 ? null : max,
    sampleCount,
  };
}

function makeRows(xs: number[], fValues: number[], derivativeValues: number[],
  exactValues: number[]): TableRow[] {
  return xs
    .filter((_, index) => index % TABLE_STEP === 0)
    .map((x, rowIndex) => {
      const sourceIndex = rowIndex * TABLE_STEP;
      return {

```

```

        x,
        fx: fValues[sourceIndex],
        derivative: derivativeValues[sourceIndex],
        exact: exactValues[sourceIndex],
    };
});
}

function makeSeries(
    settings: CalculatorSettings,
    functionValues: number[],
    derivativeValues: number[],
    exactValues: number[],
    classicalOrder: ClassicalOrder | null,
): ChartSeries[] {
    const series: ChartSeries[] = [
        {
            name: `${settings.spec.label},  $\alpha=0$ `,
            color: "#60727a",
            dashed: true,
            values: functionValues,
        },
        {
            name: `D${formatOrder(settings.alpha)} (${SCHEME_SHORT[settings.scheme]})`,
            color: "#087b7b",
            strokeWidth: 3,
            values: derivativeValues,
        },
    ];

    if (exactValues.length > 0) {
        series.push({
            name: `эталон D${classicalOrder}`,
            color: "#a86805",
            dashed: true,
            values: exactValues,
        });
    }

    return series;
}

```

## B.9 web/src/App.tsx

```

import { AuthorBadge } from "../components/AuthorBadge/AuthorBadge";
import { AppHeader } from "../components/AppHeader/AppHeader";
import { ChartPanel } from "../components/ChartPanel/ChartPanel";

```

```

import { ControlsPanel } from "../components/ControlsPanel/ControlsPanel";
import { DataTable } from "../components/DataTable/DataTable";
import { MethodNotes } from "../components/MethodNotes/MethodNotes";
import { downloadCsv } from "../domain/csv";
import { makeCsvRows } from "../domain/derivativeModel";
import { useCalculatorState } from "../hooks/useCalculatorState";
import { useDerivativeModel } from "../hooks/useDerivativeModel";
import styles from "../App.module.scss";

export default function App() {
  const calculator = useCalculatorState("sin");
  const model = useDerivativeModel(calculator.settings);

  function handleCsvDownload() {
    downloadCsv(`${calculator.settings.spec.key}_a_${calculator.settings.alpha}.csv`,
      makeCsvRows(model));
  }

  return (
    <main className={styles.shell}>
      <AppHeader />
      <section className={styles.workspace} aria-label="Расчёт дробной производной">
        <ControlsPanel calculator={calculator} onCsvDownload={handleCsvDownload} />
        <ChartPanel settings={calculator.settings} model={model} />
        <DataTable settings={calculator.settings} model={model} />
      </section>
      <MethodNotes />
      <AuthorBadge />
    </main>
  );
}

```

## Б.10 tests/conftest.py

```

import os
import sys

ROOT_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

if ROOT_DIR not in sys.path:
    sys.path.insert(0, ROOT_DIR)

```

## Б.11 tests/test\_coefficients.py

```

import numpy as np

from fractional import coefficient_partial_sums, gl_coefficients

def test_coefficients_recurrence():
    c = gl_coefficients(0.5, 4)
    expected = np.array([1.0, -0.5, -0.125, -0.0625, -0.0390625])
    assert np.allclose(c, expected)

def test_integer_order_coefficients_truncate():
    c1 = gl_coefficients(1.0, 5)
    assert np.allclose(c1, [1.0, -1.0, 0.0, 0.0, 0.0, 0.0])
    c2 = gl_coefficients(2.0, 5)
    assert np.allclose(c2, [1.0, -2.0, 1.0, 0.0, 0.0, 0.0])

def test_partial_sums_go_to_zero():
    sums = coefficient_partial_sums(0.5, 20000)
    assert abs(sums[-1]) < 0.01
    assert abs(sums[-1]) < abs(sums[100])

```

## B.12 tests/test\_fractional\_orders.py

```

import numpy as np

import functions as fn
from fractional import gl_point_from_zero

def test_power_fraction_matches_gamma():
    spec = fn.get("x2")
    val = gl_point_from_zero(spec.f, 2.0, 0.5, 0.005)
    ref = float(spec.exact_frac(np.array([2.0]), 0.5)[0])
    assert abs(val - ref) < 0.01

def test_first_order_convergence_power_frac():
    spec = fn.get("x2")
    ref = float(spec.exact_frac(np.array([2.0]), 0.5)[0])
    e1 = abs(gl_point_from_zero(spec.f, 2.0, 0.5, 0.02) - ref)
    e2 = abs(gl_point_from_zero(spec.f, 2.0, 0.5, 0.01) - ref)
    assert 1.7 < e1 / e2 < 2.3

```

```

def test_from_zero_uses_floor_grid_count():
    calls = []

    def f(x):
        calls.extend(np.asarray(x, dtype=float).tolist())
        return np.asarray(x, dtype=float)

    gl_point_from_zero(f, 1.05, 1.0, 0.1)

    assert len(calls) == 11
    assert np.isclose(calls[-1], 0.05)

```

## B.13 tests/test\_function\_domains.py

```

import numpy as np

import functions as fn

def test_ln_out_of_domain_is_nan():
    spec = fn.get("ln")
    out = spec.f(np.array([-1.0, 0.0, 1.0]))
    assert np.isnan(out[0]) and np.isnan(out[1])
    assert not np.isnan(out[2])

```

## B.14 tests/test\_integer\_orders.py

```

import numpy as np

import functions as fn
from fractional import gl_grid, gl_point

def test_alpha0_is_identity():
    spec = fn.get("sin")
    xs = np.linspace(0.5, 6.0, 25)
    got = gl_grid(spec.f, xs, 0.0, 0.01, 500)
    assert np.allclose(got, spec.f(xs))

def test_alpha1_sin_matches_cos():
    spec = fn.get("sin")
    val = gl_point(spec.f, 1.0, 1.0, 0.01, 2)
    assert abs(val - np.cos(1.0)) < 0.01

```



```

def test_alpha2_x2_is_exact():
    spec = fn.get("x2")
    val = gl_point(spec.f, 2.3, 2.0, 0.01, 500)
    exact = float(spec.classical[2](np.array([2.3]))[0])
    assert abs(val - exact) < 1e-9

def test_first_order_convergence_sin():
    spec = fn.get("sin")
    exact = np.cos(1.0)
    e1 = abs(gl_point(spec.f, 1.0, 1.0, 0.02, 50) - exact)
    e2 = abs(gl_point(spec.f, 1.0, 1.0, 0.01, 50) - exact)
    assert 1.7 < e1 / e2 < 2.3

```

## B.15 tests/test\_alt\_schemes.py

```

import numpy as np
from scipy.special import gamma

import functions as fn
from fractional import (gl_shifted_point_from_zero, gl_point_from_zero,
                        caputo_l1_point)

def test_shifted_matches_gamma():
    spec = fn.get("x2")
    ref = float(spec.exact_frac(np.array([2.0]), 0.5)[0])
    val = gl_shifted_point_from_zero(spec.f, 2.0, 0.5, 0.002)
    assert abs(val - ref) < 0.02

def test_shifted_is_first_order():
    spec = fn.get("x2")
    ref = float(spec.exact_frac(np.array([2.0]), 0.5)[0])
    e1 = abs(gl_shifted_point_from_zero(spec.f, 2.0, 0.5, 0.02) - ref)
    e2 = abs(gl_shifted_point_from_zero(spec.f, 2.0, 0.5, 0.01) - ref)
    assert 1.7 < e1 / e2 < 2.3

def test_caputo_of_constant_is_zero():
    spec = fn.get("one")
    assert abs(caputo_l1_point(spec.f, 2.0, 0.5, 0.01)) < 1e-9

def test_caputo_matches_gamma_for_power():
    spec = fn.get("x2")

```

```

    ref = float(spec.exact_frac(np.array([2.0]), 0.5)[0])
    val = caputo_l1_point(spec.f, 2.0, 0.5, 0.005)
    assert abs(val - ref) < 0.01

def test_gl_minus_caputo_is_lower_terminal_term():
    spec = fn.get("exp")
    x, alpha, h = 1.5, 0.5, 0.002
    gap = gl_point_from_zero(spec.f, x, alpha, h) - caputo_l1_point(spec.f, x, alpha, h)
    corr = 1.0 * x ** (-alpha) / gamma(1.0 - alpha)
    assert abs(gap - corr) < 0.02

```

## Б.16 tests/test\_master\_table.py

```

import numpy as np
from scipy.special import gamma

import experiments as ex

def test_master_table_power_cell_matches_gamma():
    mt = ex.master_table(0.01, 500)
    row = mt[mt["функция"] == "x^2"].iloc[0]
    exact_half = gamma(3.0) / gamma(2.5) * 2.0 ** 1.5
    assert abs(row["D^0.5"] - exact_half) < 0.02
    assert abs(row["D^2"] - 2.0) < 1e-6

def test_master_table_constant_is_memory_term():
    mt = ex.master_table(0.01, 500)
    row = mt[mt["функция"] == "1"].iloc[0]
    exact = 2.0 ** (-0.5) / gamma(0.5)
    assert abs(row["D^0.5"] - exact) < 0.02

```

## Приложение В Дополнительные таблицы

Полные таблицы значений сохраняются программой в папку `tables/`. Здесь приведены ещё две: единый расчёт всех функций при  $a = 0$  и подробные значения  $x^2$ .

Сводная таблица 7 собирает функции в разных режимах. Чтобы показать, что это выбор подачи и метод одинаково считает любую функцию,

те же двенадцать функций посчитаны единообразно, с общим нижним пределом  $a = 0$  (таблица 24); параметры и опорные точки взяты те же.

Таблица 24 – Те же функции, что в таблице 7, но в едином режиме  $a = 0$  ( $N = \lfloor x_0/h \rfloor$ ,  $h = 0,01$ )

| $f(x)$            | $x_0$ | $D^0$ | $D^{0,5}$ | $D^1$  | $D^{1,5}$ | $D^2$  |
|-------------------|-------|-------|-----------|--------|-----------|--------|
| 1                 | 2     | 1,000 | 0,399     | 0,000  | −0,100    | 0,000  |
| $x$               | 2     | 2,000 | 1,595     | 1,000  | 0,400     | 0,000  |
| $x^2$             | 2     | 4,000 | 4,247     | 3,990  | 3,186     | 2,000  |
| $x^3$             | 2     | 8,000 | 10,181    | 11,940 | 12,694    | 11,940 |
| $e^x$             | 1     | 2,718 | 2,847     | 2,705  | 2,553     | 2,691  |
| $\sin x$          | 1     | 0,841 | 0,846     | 0,545  | −0,097    | −0,836 |
| $\cos x$          | 1     | 0,540 | −0,104    | −0,839 | −1,130    | −0,549 |
| $\sin x + \cos x$ | 1     | 1,382 | 0,742     | −0,294 | −1,227    | −1,385 |
| $\ln x^*$         | 8     | —     | —         | —      | —         | —      |
| $x \sin x$        | 1     | 0,841 | 1,178     | 1,381  | 1,174     | 0,270  |
| $xe^x$            | 1     | 2,718 | 3,983     | 5,396  | 6,785     | 8,047  |
| $\tan x$          | 1     | 1,557 | 2,287     | 3,373  | 5,418     | 10,125 |

\* Для  $\ln x$  режим  $a = 0$  неприменим: история  $[0, x_0]$  задевает точку  $x = 0$ , где логарифм не определён, и сумма обращается в NaN.

Для целых порядков значения совпадают со сводной таблицей, ведь при  $\alpha = 1$  и  $\alpha = 2$  результат локален и от нижнего предела не зависит. Дробные же столбцы у  $\sin x$ ,  $\cos x$  и  $e^x$  в режиме  $a = 0$  заметно отходят от фазовых формул (12). При  $\alpha = 0,5$  численный  $\sin x$  даёт 0,846 вместо лиувиллевских 0,977,  $\cos x$  даёт −0,104 вместо −0,213,  $e^x$  даёт 2,847 вместо 2,718. Зазор здесь порядка 0,1, на порядок больше процентного зазора при конечной памяти. Причина в том, что при  $a = 0$  у этих функций появляется начальный переходный член, и фазовый сдвиг перестаёт быть точным эталоном. Поэтому в сводной таблице 7 они и взяты с конечной памятью, ближе к форме Лиувилля.

Ниже приведены значения дробной производной  $x^2$  в режиме  $a = 0$  (таблица 25).

В столбце  $D^2$  всюду ровно 2, поскольку вторая разность квадратичной функции точна. В столбце  $D^1$  значения равны  $2x - 0,01$ . Столбец  $D^{0,5}$  совпадает с формулой (11) с ошибкой порядка  $h$ . При  $x = 1$  значение 1,499 против точного  $\approx 1,505$ , при  $x = 2$  оно равно 4,247 против 4,255.

Таблица 25 – Значения дробной производной  $x^2$  в режиме  $a = 0$   
 ( $N = \lfloor x/h \rfloor$ ,  $h = 0,01$ )

| $x$ | $f(x)$ | $D^{0,5}$ | $D^1$ | $D^{1,5}$ | $D^2$ |
|-----|--------|-----------|-------|-----------|-------|
| 1,0 | 1,00   | 1,499     | 1,99  | 2,248     | 2,00  |
| 2,0 | 4,00   | 4,247     | 3,99  | 3,186     | 2,00  |
| 3,0 | 9,00   | 7,808     | 5,99  | 3,904     | 2,00  |
| 4,0 | 16,00  | 12,025    | 7,99  | 4,509     | 2,00  |
| 5,0 | 25,00  | 16,808    | 9,99  | 5,042     | 2,00  |